[BUSINESS] FORECASTING AND PREDICTIVE MODELING

LECTURE 11

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 10 ...

- ARIMA models (seasonal and non-seasonal)

Today

- Dynamic regression models

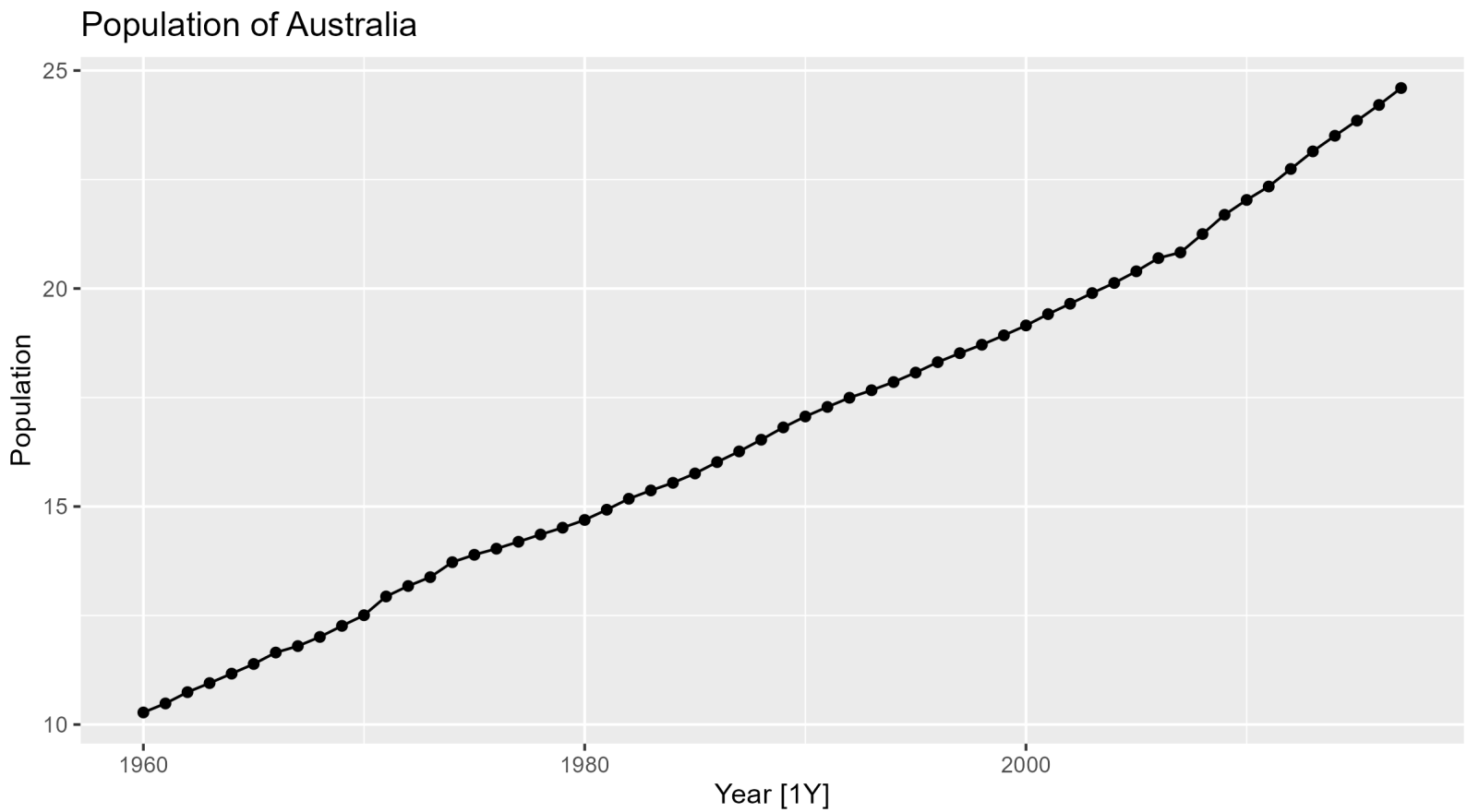
Comparing ETS and ARIMA models I

- How do you compare the performance of different models?
- Assume that you want to forecast Australia's population using ETS and ARIMA.

```
aus_population <- global_economy |>
  filter(Code == "AUS") |>
  mutate(Population = Population / 1e6) |> # to have smaller values
  select(Population)

aus_population |>
  autoplot() +
  geom_point() +
  labs(title = "Population of Australia")
```

Comparing ETS and ARIMA models II



Comparing ETS and ARIMA models III

```
aus_population |>
  model(ets = ETS(Population),
        arima = ARIMA(Population)) -> population_models

population_models |>
  glance()
```

```
# A tibble: 2 × 11
  .model  sigma2 log_lik    AIC   AICc    BIC      MSE    AMSE
  <chr>    <dbl>   <dbl>  <dbl>  <dbl>  <dbl>    <dbl>   <dbl>
1 ets    0.00413    43.5  -77.0  -75.8  -66.7  0.00385  0.0120
2 arima  0.00406    74.9 -146.  -146.  -142.   NA       NA
```

- Both AIC and AICc are smaller for ARIMA. Does this mean that ARIMA is better?

Comparing ETS and ARIMA models IV

- When we want to compare different models (ETS, ARIMA) for a time series it is important to remember that AICc cannot be used for comparison. It can be used to compare models from the same method (although, in case of ARIMA, the differencing needs to be the same).
- We need to use accuracy measures (preferably with cross-validation). Let's see two examples.
- For a non-seasonal example, let's consider Australia's population and do a time series cross validation.

Comparing ETS and ARIMA models V

```
aus_popuation |>
  stretch_tsibble(.init = 10) |>
  model(ets = ETS(Population),
        arima = ARIMA(Population)) -> population_models

models |> view()
models |> tidy()

models |>
  forecast(h = 1) |>
  accuracy(aus_population) |>
  select(.model, .type, RMSE, MAE, MAPE)
```

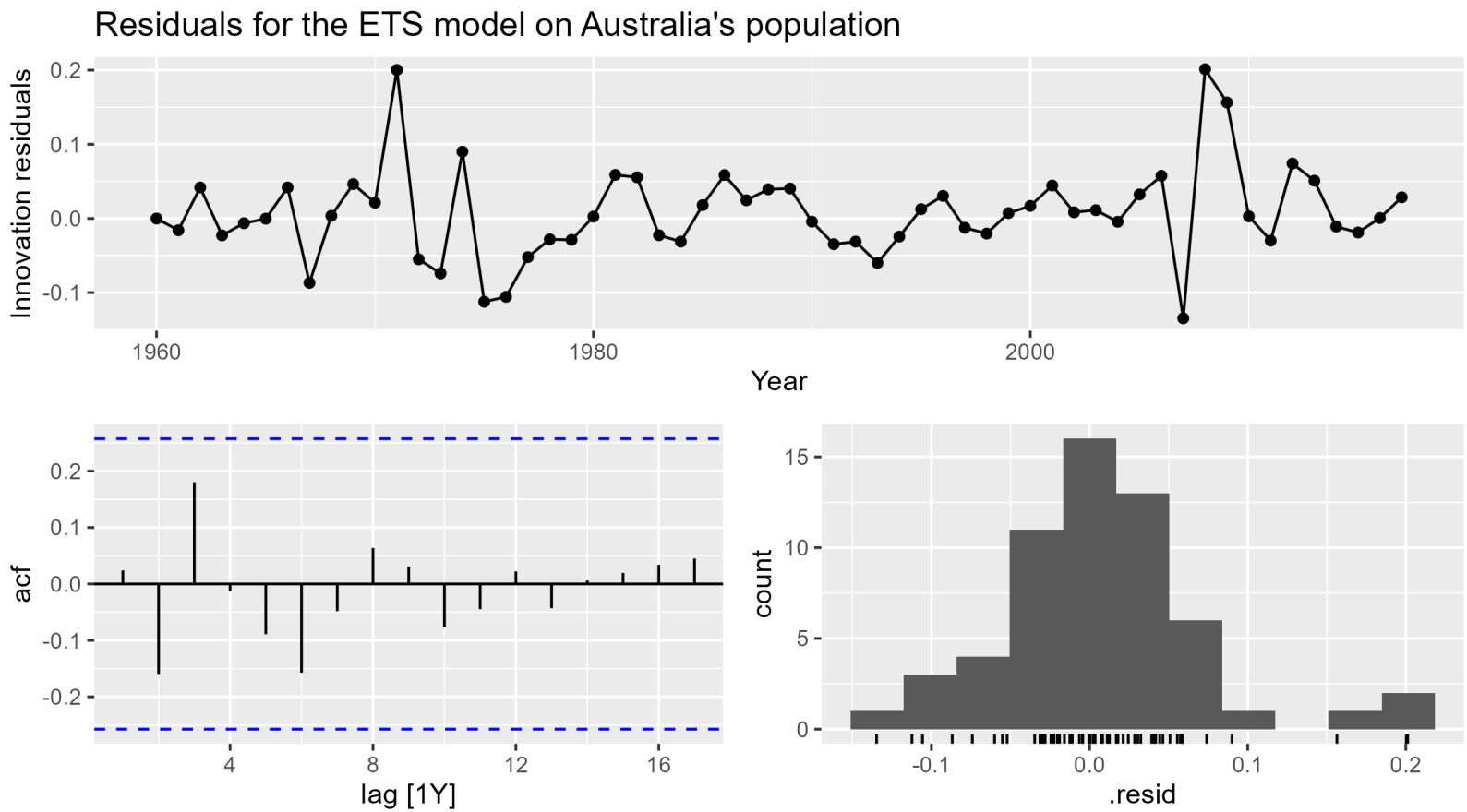
```
  .model .type   RMSE    MAE  MAPE
  <chr>  <chr>  <dbl>  <dbl> <dbl>
1 arima  Test  0.194  0.0789 0.509
2 ets    Test  0.0774 0.0543 0.327
```

Comparing ETS and ARIMA models VI

```
#let's look at the residuals
aus_population |>
  model(ETS(Population)) -> ets_model

ets_model |>
  gg_tsresiduals() +
  labs(title = "Residuals for the ETS model on Australia's population")
```

Comparing ETS and ARIMA models VII



Comparing ETS and ARIMA models VIII

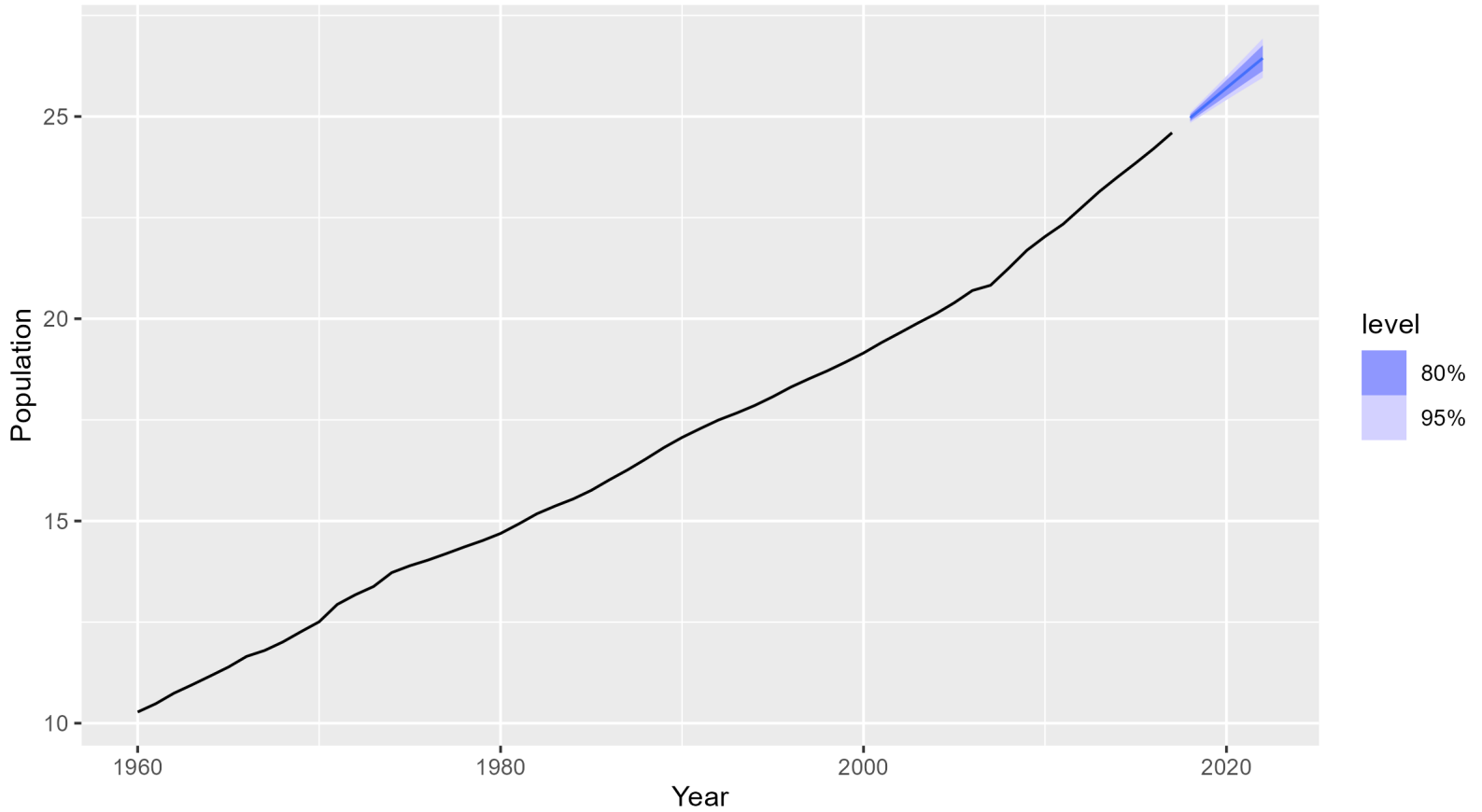
```
ets_model |>
  augment() |>
  features(.innov, ljung_box) #high p-value means white noise
```

```
  .model           lb_stat lb_pvalue
  <chr>              <dbl>     <dbl>
1 ETS(Population)  0.0349     0.852
```

- The ETS model seems better according to all performance measures and the residual diagnostics look OK as well. If we use it to forecast for the next 5 years we will get:

```
aus_population |>
  model(ETS(Population)) |>
  forecast(h = 5) |>
  autoplot(aus_population)
```

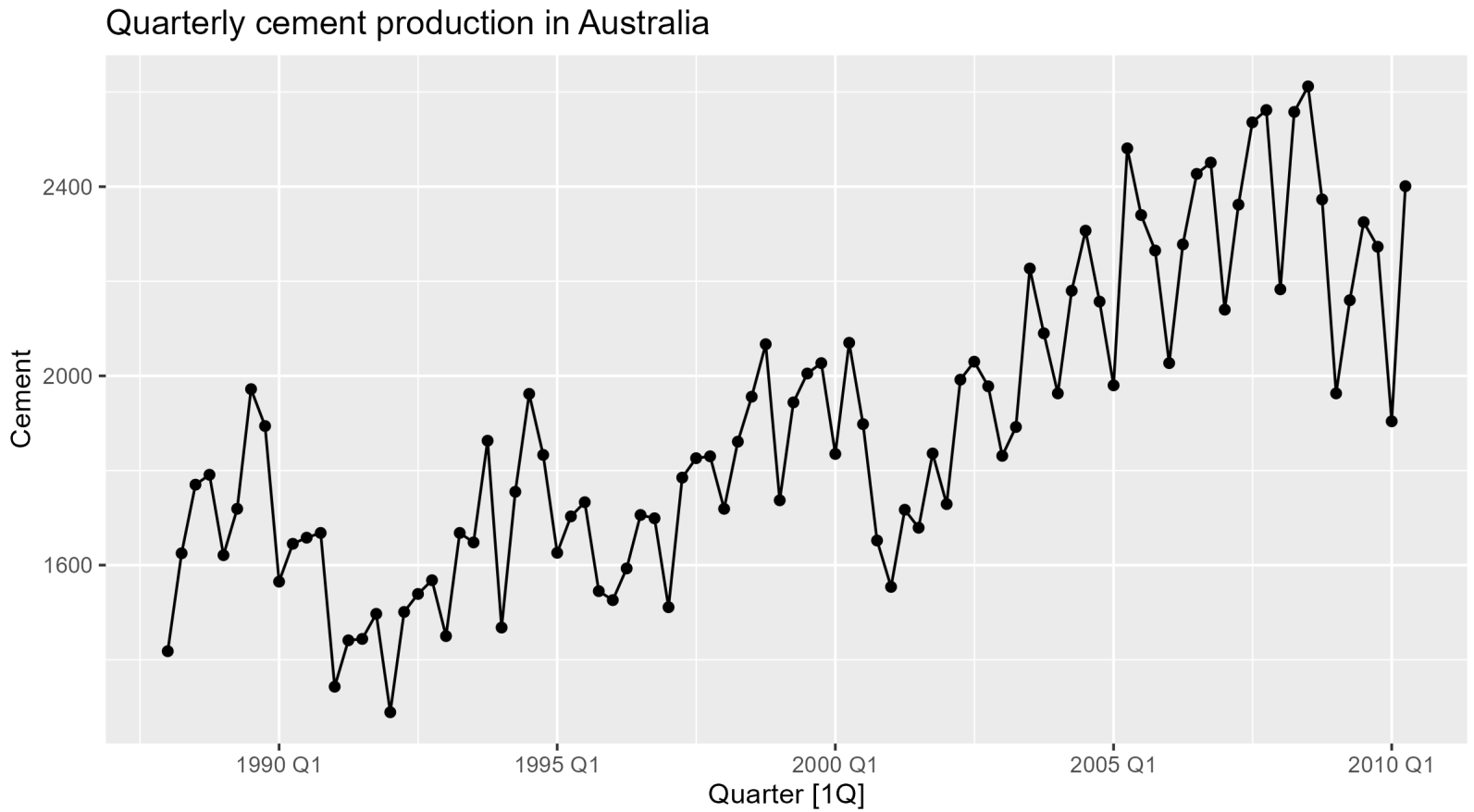
Comparing ETS and ARIMA models IX



- As a seasonal example, we will consider the quarterly cement production data (part of the *aus_production* data set).
- This is a longer time series, 218 observations (compared to 58 observations for Australian population), we will only use data from 1988 (90 observations) and split it into train-test set (80 observations train - 10 observations test).
- We will fit separately an ARIMA (with automatically found parameters) and an ETS model.

```
cement_production <- aus_production |>
  select(Cement) |>
  filter(year(Quarter) >= 1988)

cement_production |>
  autoplot() +
  geom_point() +
  labs(title = "Quarterly cement production in Australia")
```


Quarterly cement production in Australia
Cement
2400
2000
1600
1990 Q1
1995 Q1
2000 Q1
2005 Q1
2010 Q1
Quarter [1Q]

```r
cement_train <- cement |> head(n = 80)
cement_test <- cement |> tail(n = 10)

cement_models <-
  cement_train |>
  model(arima = ARIMA(Cement),
        ets = ETS(Cement))

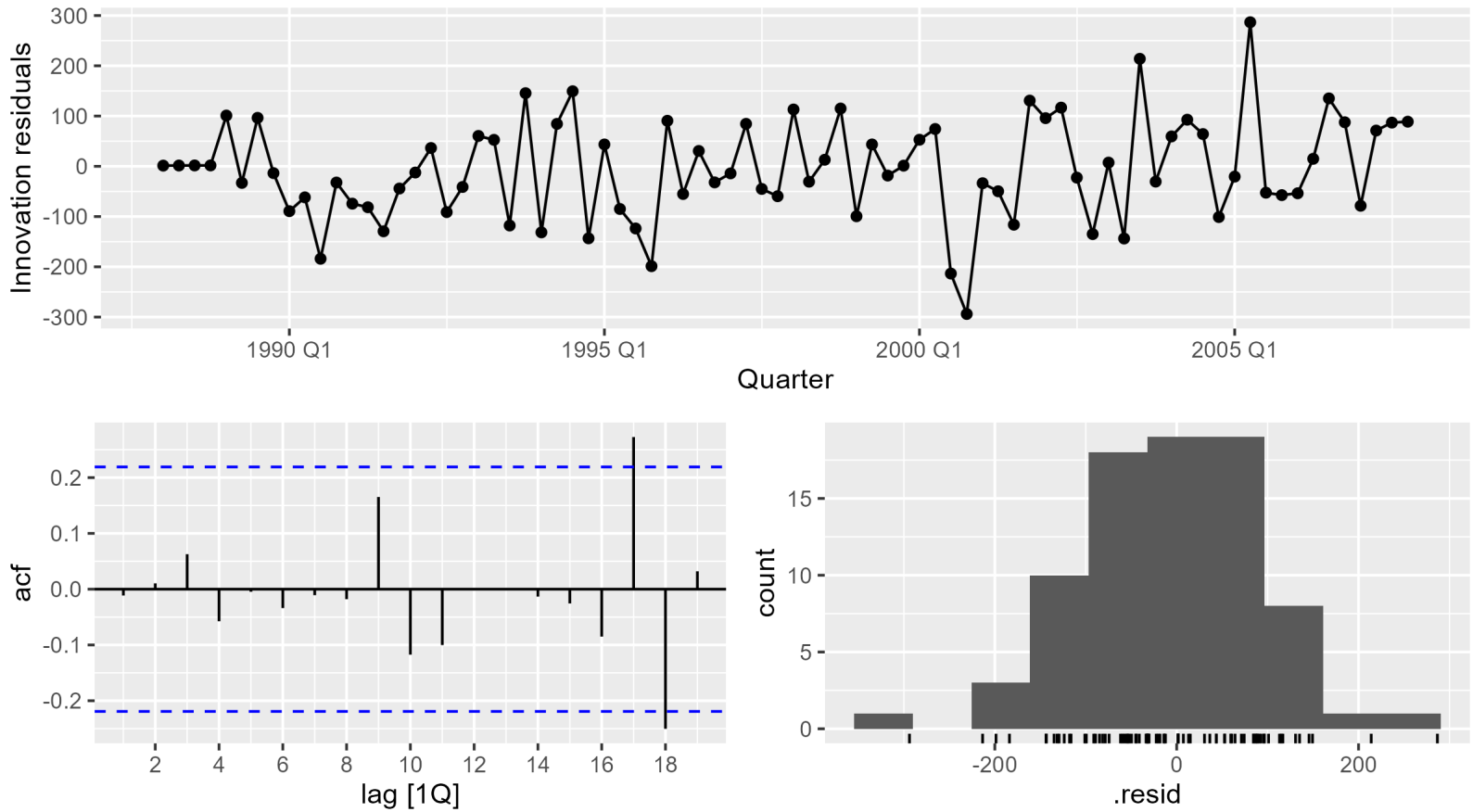
cement_models[1] |> report()
```

```
Series: Cement
Model: ARIMA(1,0,1)(2,1,1)[4] w/ drift

Coefficients:
         ar1      ma1   sar1     sar2     sma1  constant
      0.8886  -0.2366  0.081  -0.2345  -0.8979    5.3884
s.e.  0.0842   0.1334  0.157   0.1392   0.1780    1.4844

sigma^2 estimated as 11456:  log likelihood=-463.52
AIC=941.03   AICc=942.68   BIC=957.35
```

```
cement_models[1] |> gg_tsresiduals()
```



```
cement_models[1] |>
  augment() |>
  features(.innov, ljung_box, lag = 8, dof = 5) #white noise
```

```
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 arima    0.783     0.853
```

```
cement_models[2] |> report()
```

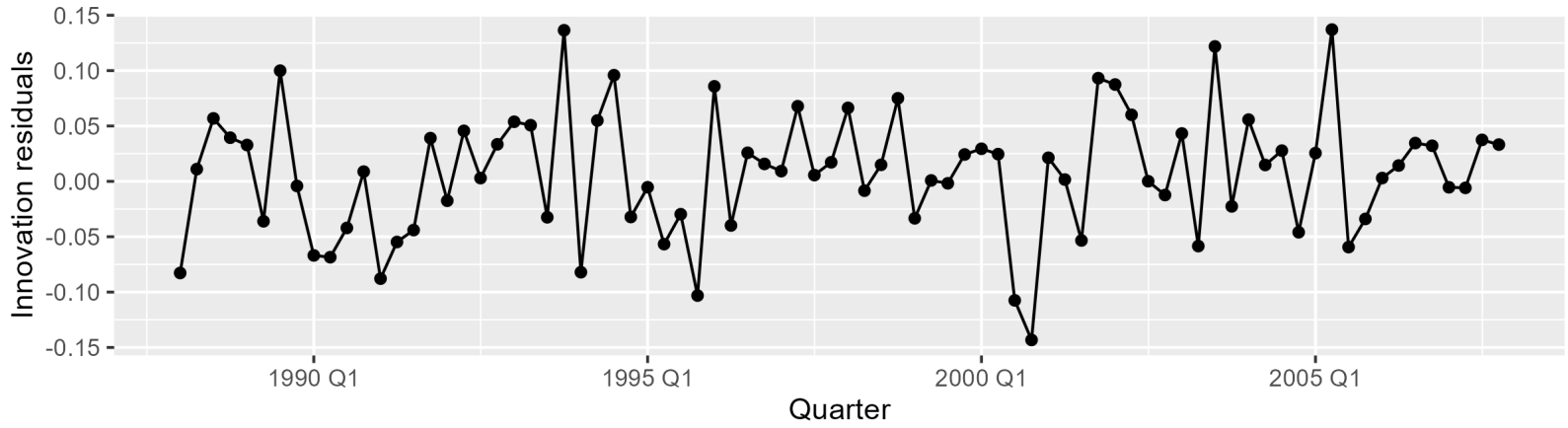
```
Series: Cement
Model: ETS(M,N,M)
  Smoothing parameters:
    alpha = 0.7533714
    gamma = 0.0001000093

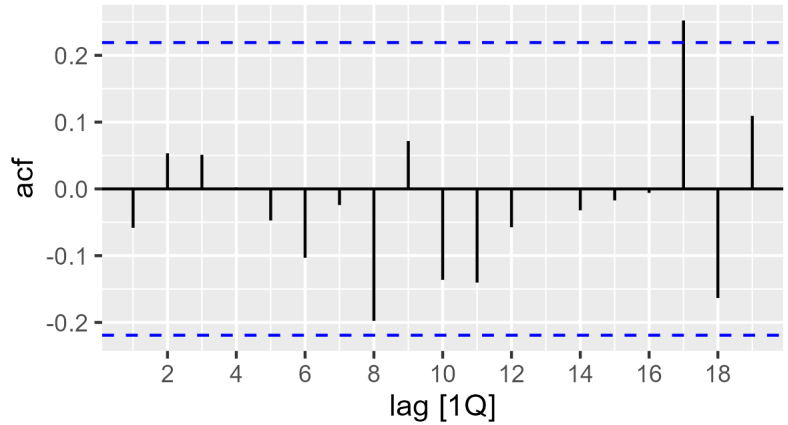
  Initial states:
     l[0]     s[0]    s[-1]    s[-2]     s[-3]
 1694.712 1.031179 1.045209 1.011424 0.9121874

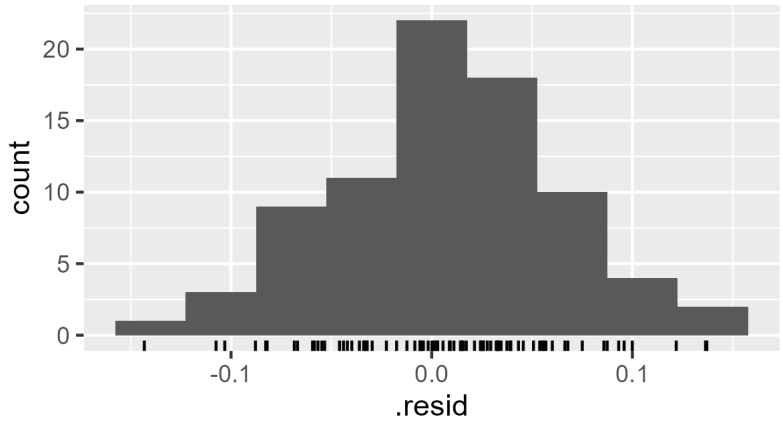
  sigma^2:  0.0034

     AIC     AICc      BIC
1104.095 1105.650 1120.769
```

```
cement_models[2] |> gg_tsresiduals()
```


Innovation residuals
0.15
0.10
0.05
0.00
-0.05
-0.10
-0.15
1990 Q1
1995 Q1
2000 Q1
2005 Q1
Quarter


acf
0.2
0.1
0.0
-0.1
-0.2
2
4
6
8
10
12
14
16
18
lag [1Q]


count
20
15
10
5
0
-0.1
0.0
0.1
.resid

```
cement_models[2] |>
  augment() |>
  features(.innov, ljung_box, lag = 8)
```

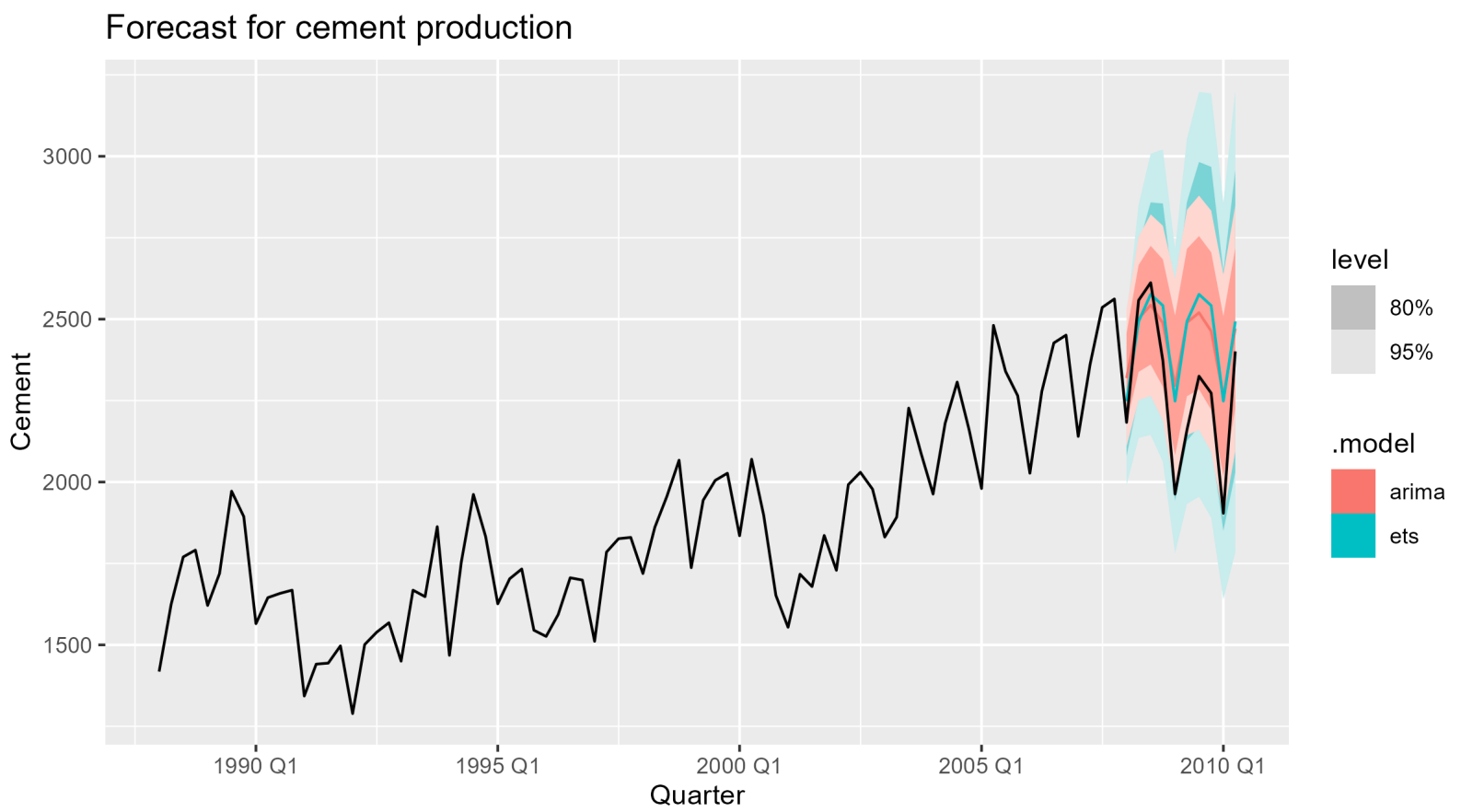
```
# A tibble: 1 × 3
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 ets       5.49     0.704
```

```
bind_rows(cement_models |> accuracy(),
          cement_models |> forecast(h = 10) |>
            accuracy(cement_test)) -> results

results |>
  select(.model, .type, RMSE, MAE, MAPE)
```

```
  .model         .type     RMSE   MAE  MAPE
  <chr>          <chr>    <dbl> <dbl> <dbl>
1 ARIMA(Cement)  Training  100.   79.9  4.37
2 ETS(Cement)    Training  103.   80.0  4.41
3 ARIMA(Cement)  Test      216. 186.   8.68
4 ETS(Cement)    Test      222. 191.   8.85
```

```
cement_models |>
  forecast(h = 10) |>
  autoplot(cement_production) +
  labs(title = "Forecast for cement production")
```



Dynamic regression models I

- Until now we have talked about two types of forecasting models:
  - regression, which allowed the inclusion of other information (other time series, effect of holidays, etc.), but was not using the fact that we deal with time series data
  - ETS and ARIMA, which were specific for time series and used past observations of the time series, but could not include other information.
- Obviously, both have advantages and we would like to combine them.

Dynamic regression models II

- Let's start from regression. Remember the regression model was the following:

$$y_t = \beta_0 + \beta_1 * x_{1,t} + ... + \beta_k * x_{k,t} + \epsilon_t$$

- where we consider k predictor variables and ϵ is the error, which we wanted to be white noise. We did a lot of residual diagnostics and Ljung-Box tests to see if it really is white noise.
- In dynamic regression models, we allow the error from the regression to contain autocorrelation and we will build an ARIMA model of the errors.
- To suggest that the error is no longer white noise, we will replace the ϵ_t notation with η_t

Dynamic regression models III

- So, for example a dynamic regression model with k predictors and ARIMA(1, 1, 1) errors will be written as:

$$y_t = \beta_0 + \beta_1 * x_{1,t} + ... + \beta_k * x_{k,t} + \eta_t$$

$$(1 - \phi_1 B)(1 - B)\eta_t = (1 + \theta_1 B)\epsilon_t$$

- where ϵ_t (the error from ARIMA) is white noise.

Dynamic regression models in R

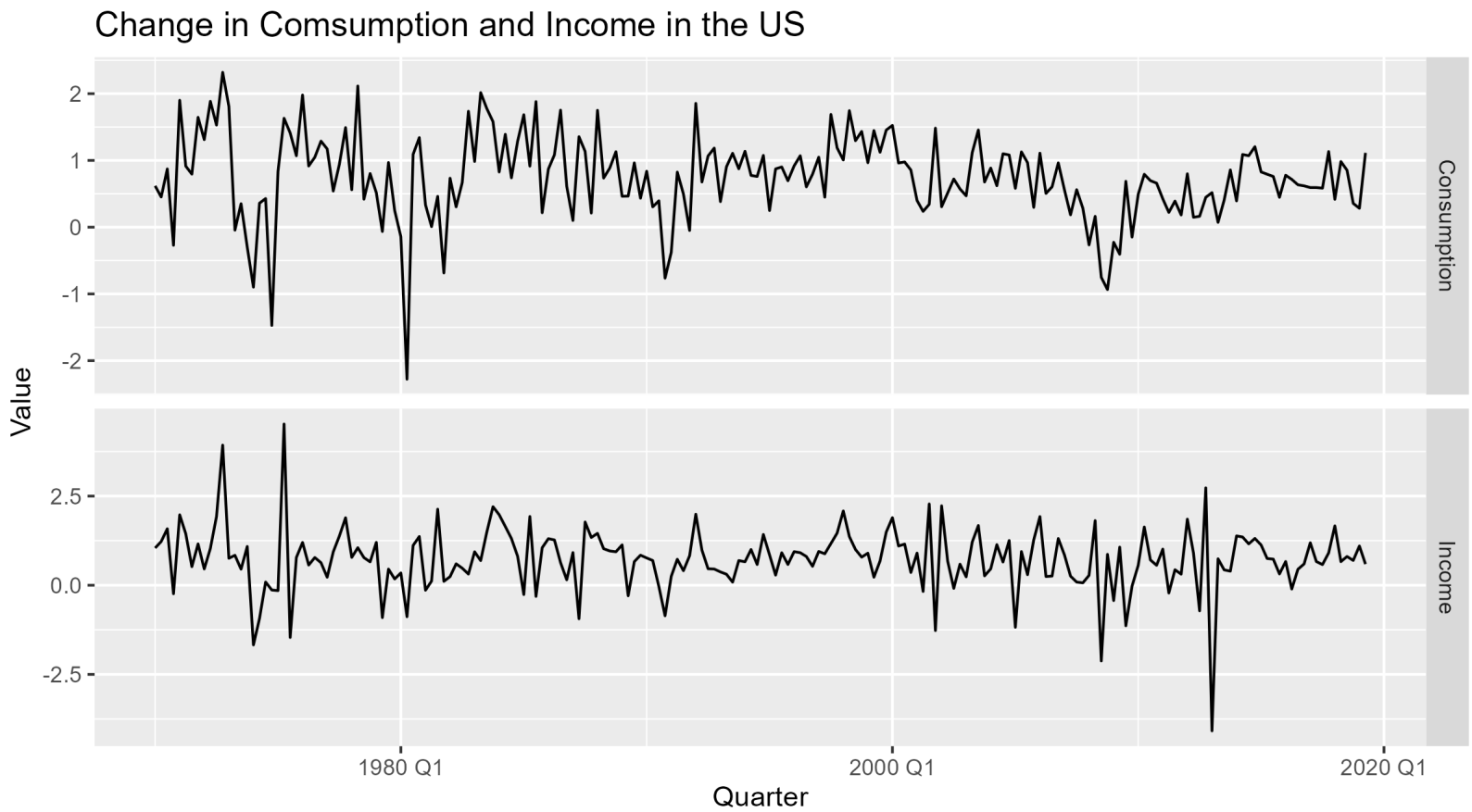
- In *R* we can define dynamic regression models with the *ARIMA* function, by adding in it other predictor variables as well.
- For example, the command: $ARIMA(y \sim x + pdq(1, 1, 0))$ will fit a regression model for y with one predictor variable (x) and an AR(1) error term.
- Since differencing is included ($d = 1$), it will be applied to all variables in the model (not just y).
- If we do not specify the *pdq* special, the *ARIMA* function will automatically find the model which best fits the errors. It determines whether differentiation is needed using the KPSS tests.
- The final model will be expressed in terms of the original variables, even if internally differentiation was used.

Example I

- Remember the US consumption data set that we used when discussing regression models? Let's use that again.
- One simple regression model that we were experimenting with was to build a model to forecast change in consumption based on change in income.

```
us_change |>
  pivot_longer(c(Consumption, Income), names_to = "Variable", values_to ="Value"
      ) |>
  ggplot(aes(x = Quarter, y = Value)) +
  geom_line() +
  facet_grid(vars(Variable), scales = "free_y")
```

Example II


Change in Comsumption and Income in the US
Value
2
1
0
-1
-2
Consumption
2.5
0.0
-2.5
Income
1980 Q1
2000 Q1
2020 Q1
Quarter

Example III

```
us_change |>
  model(reg = TSLM(Consumption ~ Income),
        arima = ARIMA(Consumption),
        dyn_reg = ARIMA(Consumption ~ Income)) -> consumption_models

consumption_models[3] |>
  report()
```

```
Series: Consumption
Model: LM w/ ARIMA(1,0,2) errors

Coefficients:
         ar1      ma1     ma2  Income  intercept
      0.7070  -0.6172  0.2066  0.1976     0.5949
s.e.  0.1068   0.1218  0.0741  0.0462     0.0850

sigma^2 estimated as 0.3113:  log likelihood=-163.04
AIC=338.07   AICc=338.51   BIC=357.8
```

Example IV

- The fitted model is (where y is consumption and x is income):

$$y_t = 0.595 + 0.198 * x_t + \eta_t$$

$$\eta_t = 0.707 * \eta_{t-1} - 0.617 * \epsilon_{t-1} + 0.207 * \epsilon_{t-2} + \epsilon_t$$

- We can check the residuals of the model using the *residual* function. However, we have two types of residuals, one for the regression model (the η_t values on which we build the ARIMA model) and the actual innovation residuals for the model (the ϵ_t values).
- The *residual* function takes as parameter the type of residual to return.

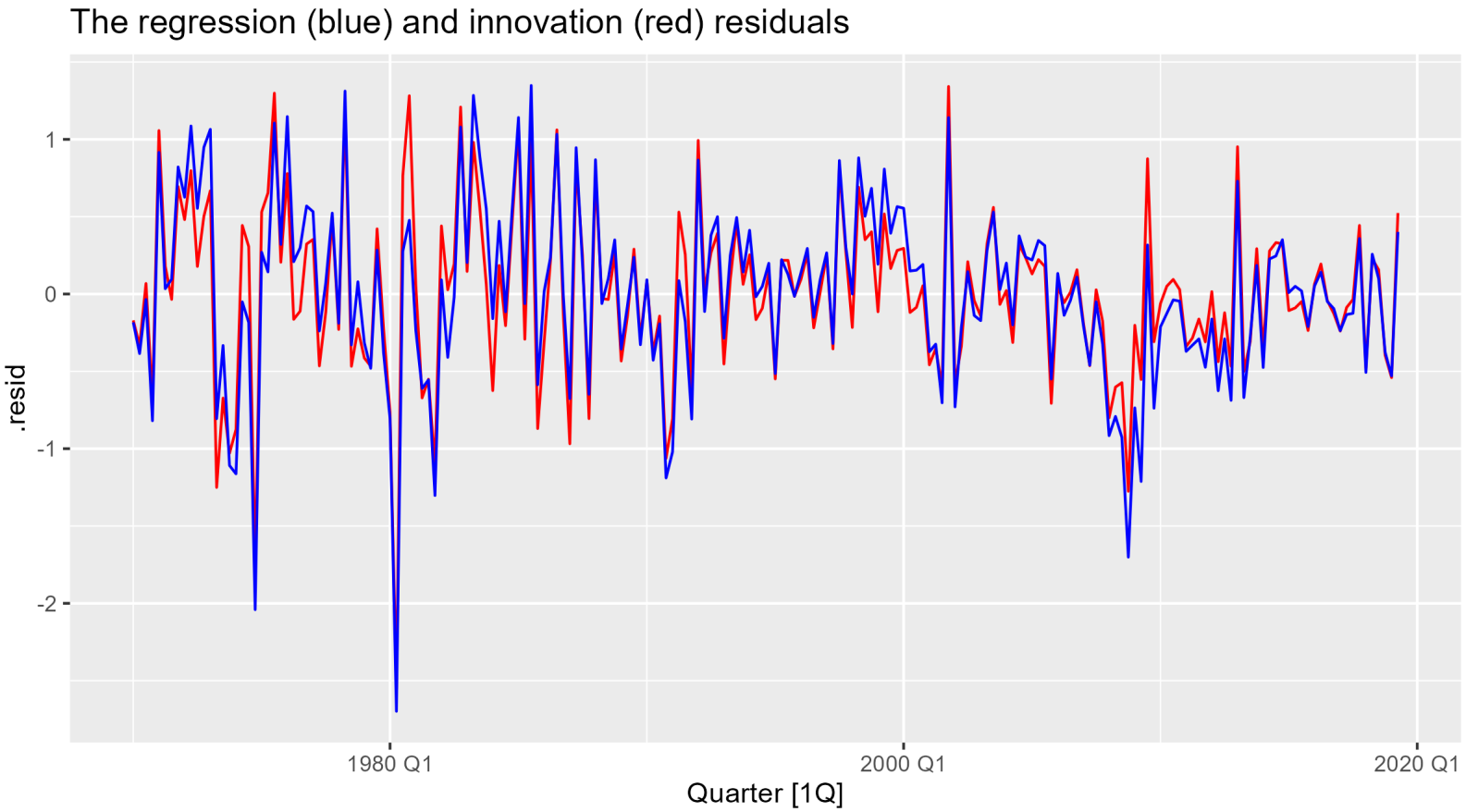
Example V

```
consumption_models[3] |>
  residuals(type = "regression")

consumption_models[3] |>
  residuals(type = "innovation")

consumption_models[3] |>
  residuals(type = "innovation") |>
  autoplot(color = "red") +
  autolayer(consumption_models[3] |> residuals(type = "regression"), color = "
      blue") +
  labs(title = "The regression (blue) and innovation (red) residuals")
```

Example VI

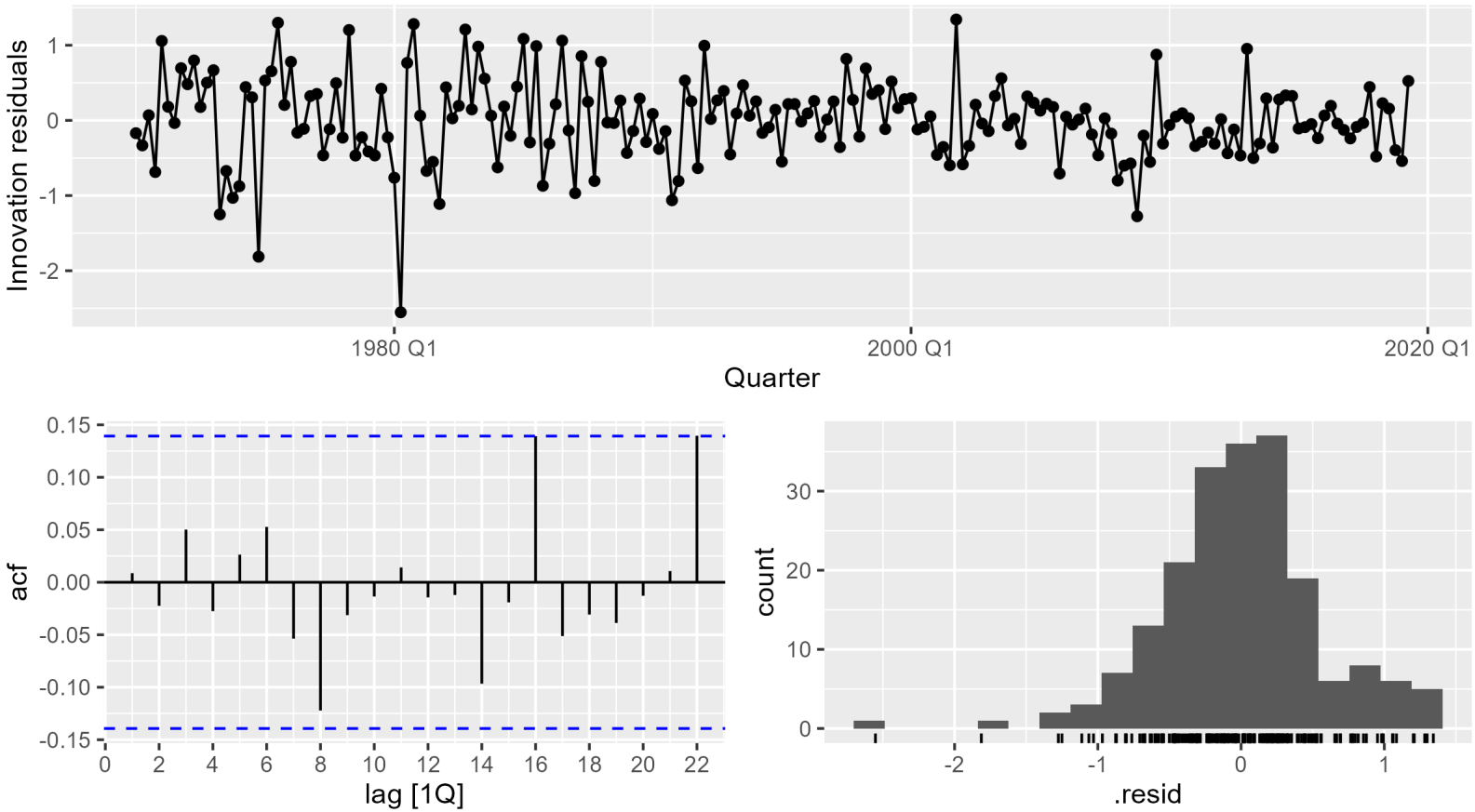


- Let's look at the residuals of the model. Only the innovation residuals should be white noise.

Example VII

```
consumption_models[3] |>
  gg_tsresiduals()
```

Example VIII



Example IX

```
consumption_models[3] |>
  augment() |>
  features(.innov, ljung_box, lag = 8, dof = 3)
```

```
# A tibble: 1 × 3
  .model  lb_stat lb_pvalue
  <chr>     <dbl>     <dbl>
1 dyn_reg    5.21     0.391
```

Forecasting with dynamic regression models

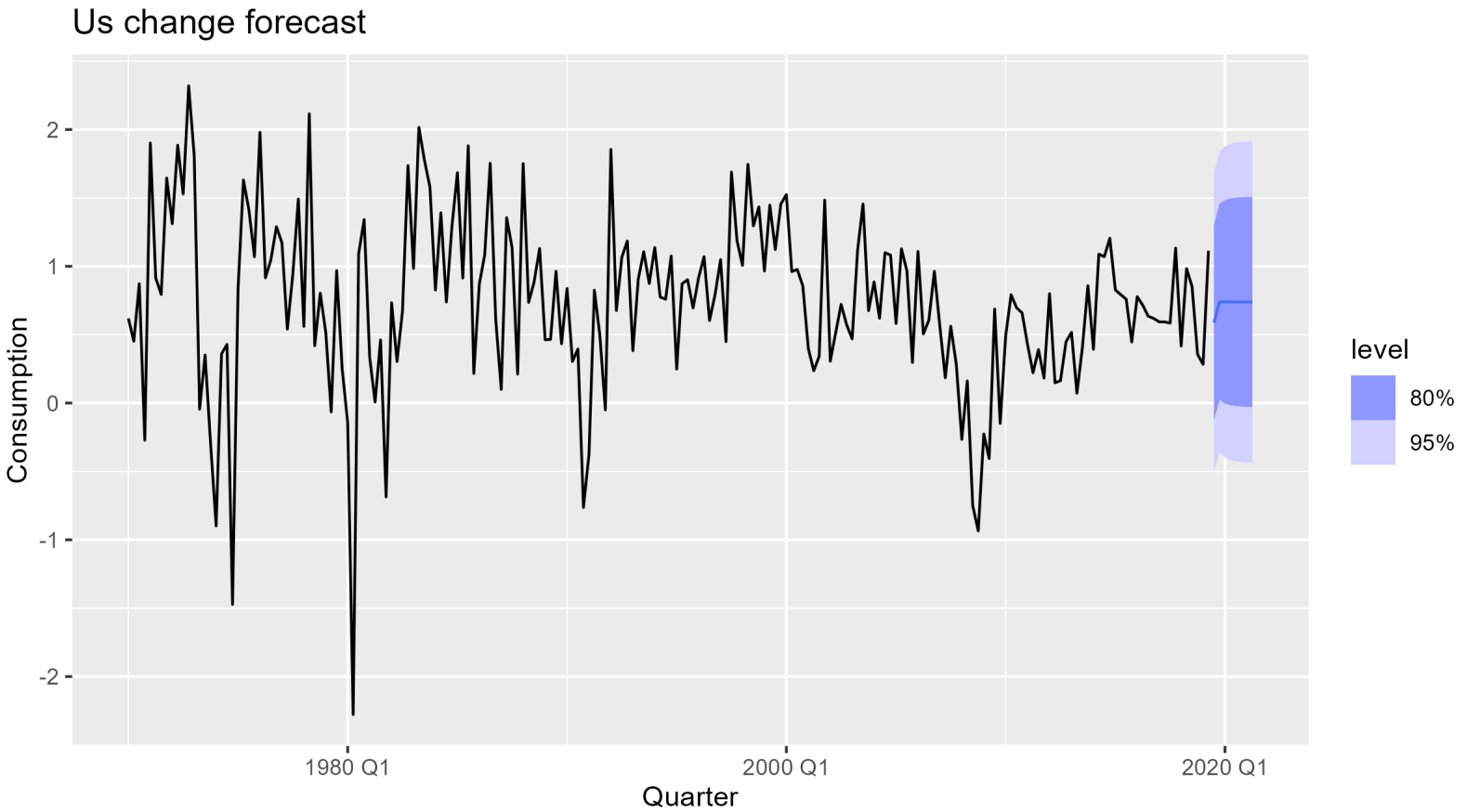
- Forecasting is simply done by forecasting separately the regression part and the ARIMA part and combining the results.
- However, in case of the regression model we first need to forecast the predictor variables. This is easy, if our predictors are calendar related (day of week, holidays, etc.), but if they are unknown, we either need to model them separately or use assumed values for them.
- Continuing our example where we forecast consumption based on income, if we want to really forecast consumption, we need to forecast the income first. Let's just assume that the income is going to be equal to the mean income of the entire time series.

Example 1 I

```
us_change_future <- us_change |>
  new_data(8)
us_change_future
us_change_future <- us_change_future |>
  mutate(Income = mean (us_change$Income))
us_change_future

consumption_model |>
  forecast(new_data = us_change_future) |>
  autoplot(us_change) +
  labs(title = "Us change forecast")
```

Example 1 II


Us change forecast
Consumption
2
1
0
-1
-2
1980 Q1
2000 Q1
2020 Q1
Quarter
level
80%
95%

Example 2 I

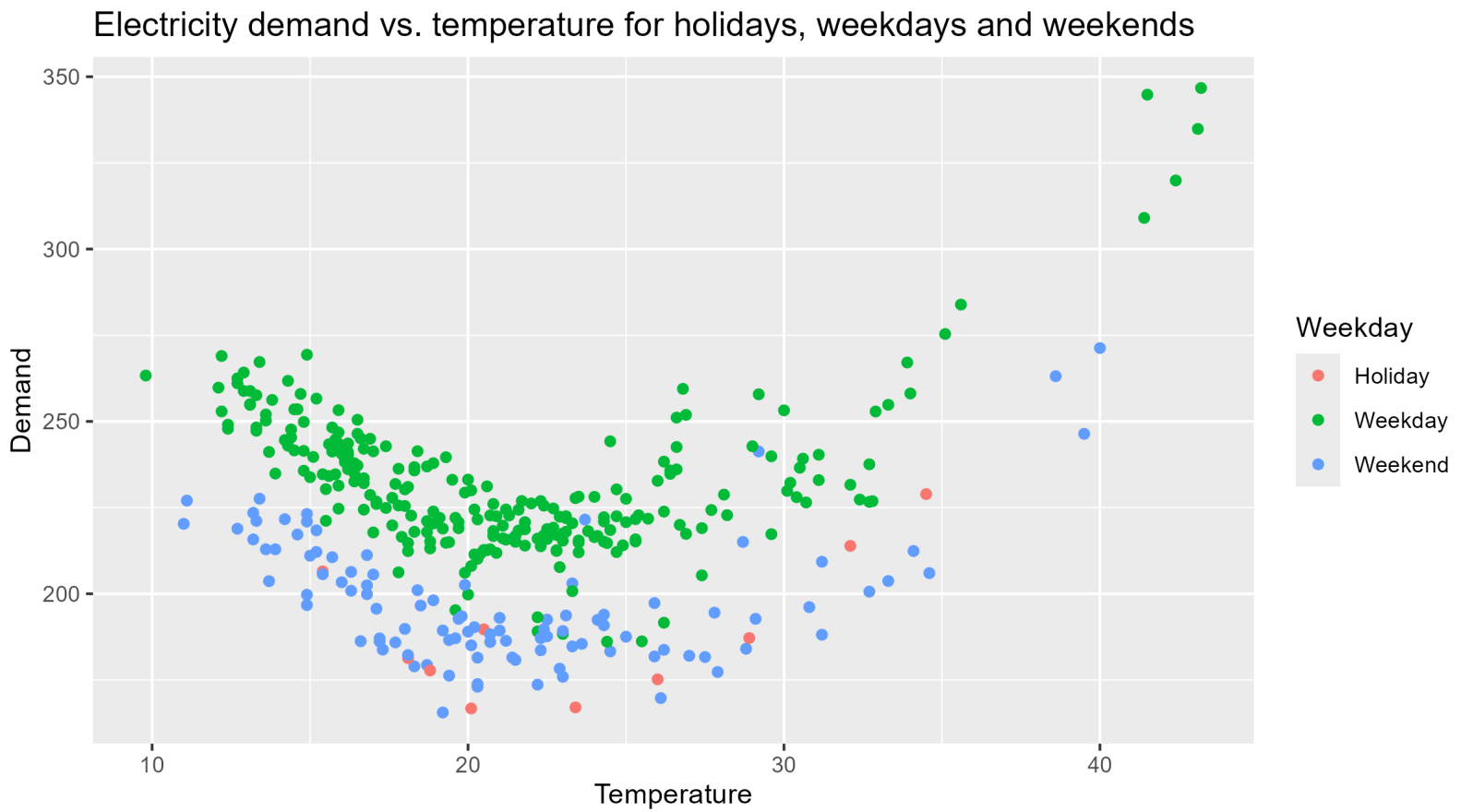
- As a second example let's use the *vic_elec* data set, containing information about temperature and electricity demand for Victoria, Australia. The tsibble contains one observation for every half an hour for three entire years.
- We will use only data from 2014 and we will aggregate it to daily data.
- The data set contains the following attributes:
  - Electricity demand - we will use its sum (for a whole day)
  - Temperature - we will use the maximum
  - Holiday (whether the day is a holiday) - this is technically equal for the entire day, we will use that value
- We will add another, dummy, attribute, *Weekday*, which will have the value "Holiday", "Weekday", "Weekend".

Example 2 II

```
vic_elec_daily <- vic_elec |>
  filter(year(Time) == 2014) |>
  index_by (Date = date(Time)) |>
  summarise(Demand = sum(Demand)/1e3,
            Temperature = max(Temperature),
            Holiday = any(Holiday)) |>
  mutate(Weekday = case_when(
         Holiday ~ "Holiday",
         wday(Date) %in% 2:6 ~ "Weekday",
         TRUE ~ "Weekend"
         )
)

vic_elec_daily |>
  ggplot(mapping = aes(x = Temperature, y = Demand, colour = Weekday)) +
  geom_point() +
  labs(title = "Electricity demand vs. temperature for holidays, weekdays and
      weekends")
```

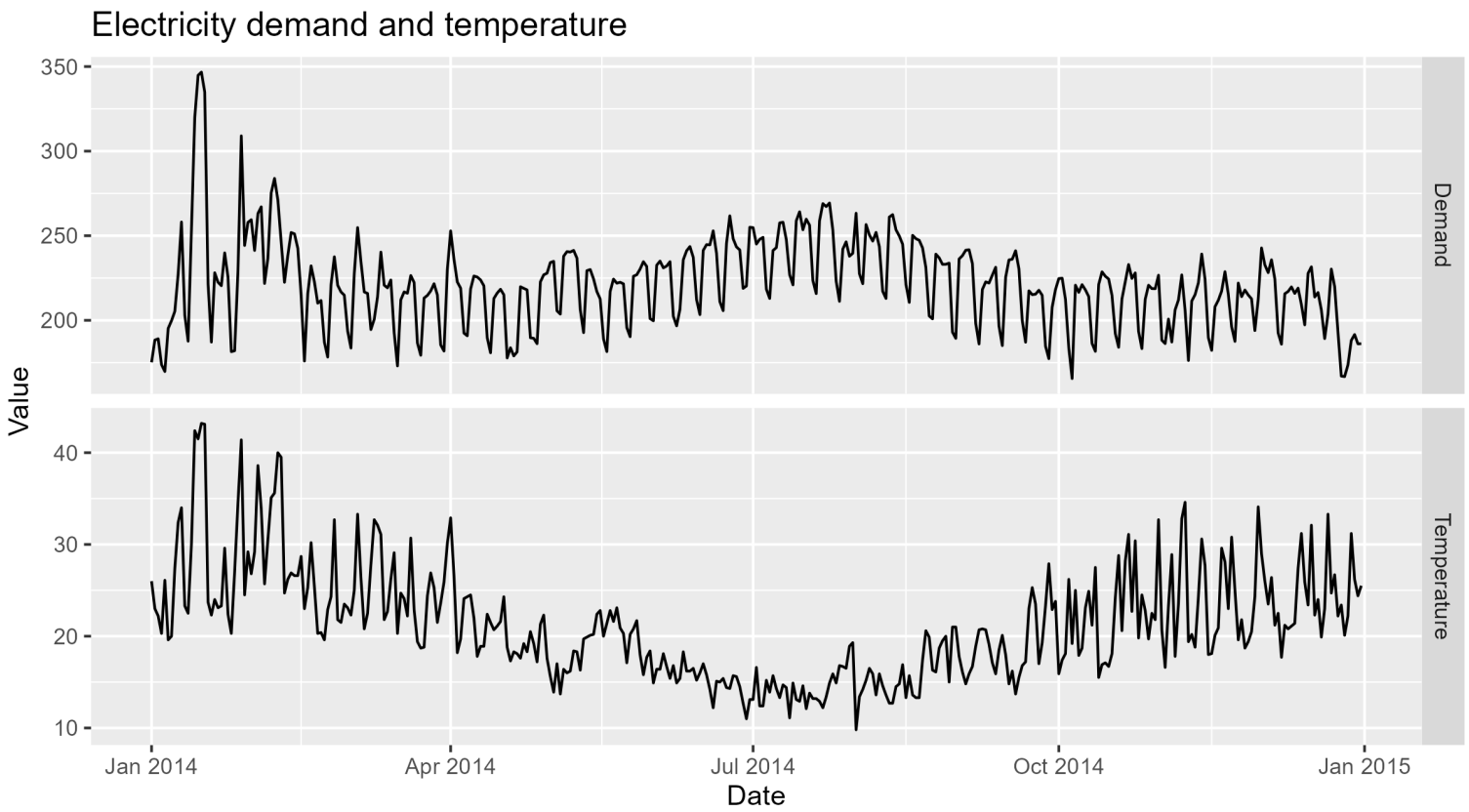
Example 2 III



Example 2 IV

```
vic_elec_daily |>
  pivot_longer(c(Temperature, Demand), values_to = "Value", names_to = "var") |>
  ggplot(mapping = aes(x = Date, y = Value)) +
  geom_line() +
  facet_grid(vars(var), scales = "free_y")
```

Example 2 V


Electricity demand and temperature
350
300
250
200
40
30
20
10
Value
Demand
Temperature
Jan 2014
Apr 2014
Jul 2014
Oct 2014
Jan 2015
Date

Example 2 VI

- From the scatterplot, it seems like there is a non-linear, probably quadratic relation between electricity demand and temperature, so we will use a quadratic model for regression. For this, we will build a separate attribute containing the square of the temperature and include it in the model.

- We will include an indicator variable as well, to see if the day is weekday or not.

```
vic_elec_daily <- vic_elec_daily |>
  mutate(TSquare = Temperature * Temperature)

vic_elec_daily_model <- vic_elec_daily |>
  model(ARIMA(Demand ~ Temperature + TSquare + (Weekday == "Weekday")))
vic_elec_daily_model |> report()
```

Example 2 VII

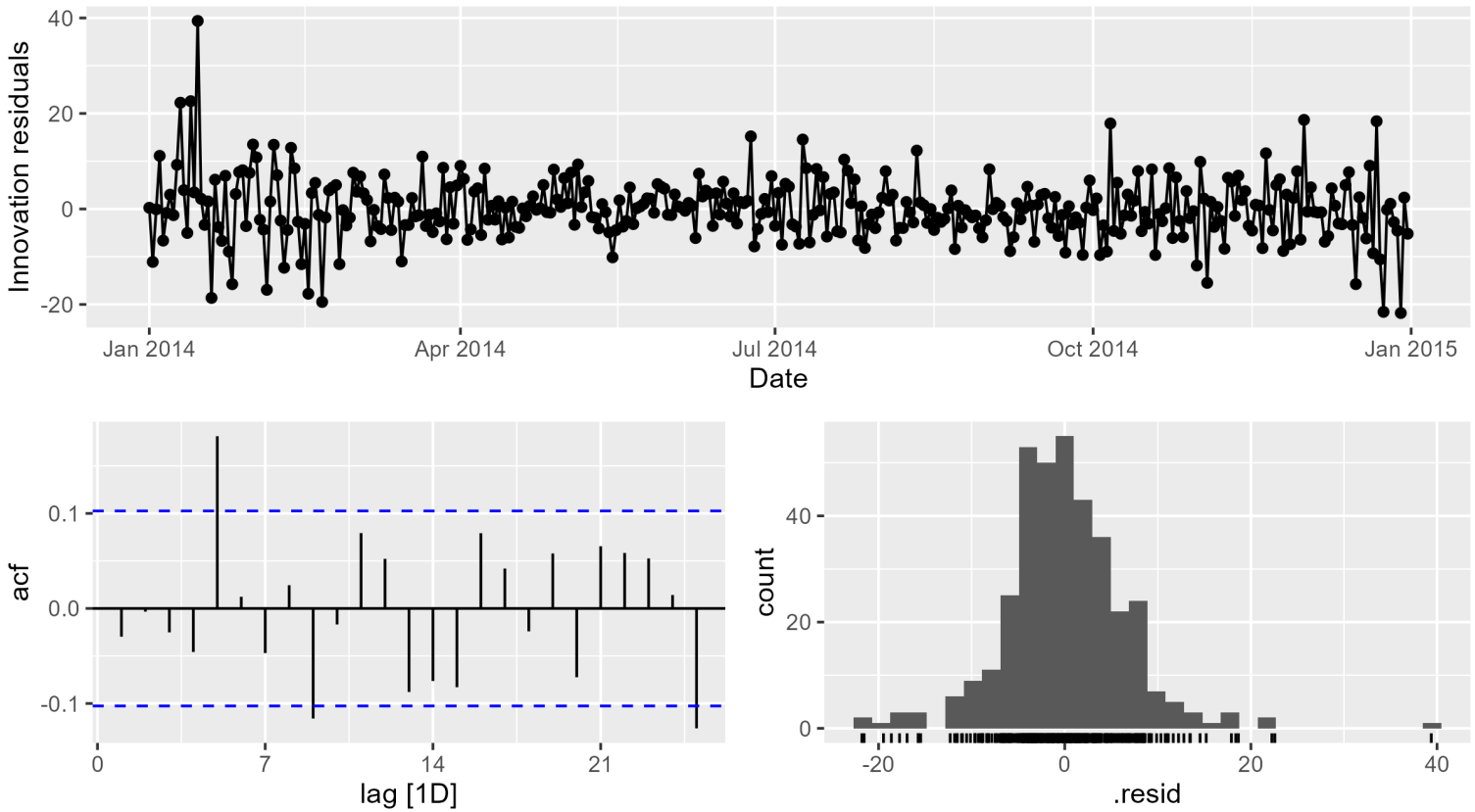
```
Series: Demand
Model: LM w/ ARIMA(2,1,2)(2,0,0)[7] errors

Coefficients:
          ar1     ar2      ma1      ma2    sar1    sar2  Temperature  TSquare  Weekday == "Weekday"TRUE
      -0.1093  0.7226  -0.0182  -0.9381  0.1958  0.4175      -7.6135   0.1810                   30.4040
s.e.   0.0779  0.0739   0.0494   0.0493  0.0525  0.0570       0.4482   0.0085                    1.3254

sigma^2 estimated as 44.91:  log likelihood=-1206.11
AIC=2432.21   AICc=2432.84   BIC=2471.18
```

Example 2 VIII

```
vic_elec_daily_model |>
  gg_tsresiduals()
```



Example 2 IX

```
vic_elec_daily_model |>
  augment() |>
  features(.innov, ljung_box, dof = 6, lag = 14)
```

```
  .model                                                                lb_stat lb_pvalue
  <chr>                                                                   <dbl>     <dbl>
1 "ARIMA(Demand ~ Temperature + TSquare + (Weekday == \"Weekday\"))"      28.4  0.000404
```

Example 2 X

- There is still autocorrelation in the residuals, and on the residual plot it can be seen that there is a higher variance in the residuals at the beginning of the year (January, February).
- **Note:** If we wanted to use this model for actual forecasting, we need information about the future values of the predictor variables. Determining whether the next days are holiday or weekday is easy, but you need a forecast for the temperature.

Lagged predictors

- Sometimes, the impact of a predictor variable cannot be seen immediately in the forecast variable, some time needs to pass. One such example is advertisements, when the effect of a campaign is likely to be seen only after some time passes.
- In such situations, we can use lagged predictors, where the predictor variables are simply lagged versions of the actual predictor. Assuming only one predictor variable, this would be:

$$y_t = \beta_0 + \gamma_1 * x_t + \gamma_2 * x_{t-1} + ... + \gamma_k * x_{t-k} + \eta_t$$

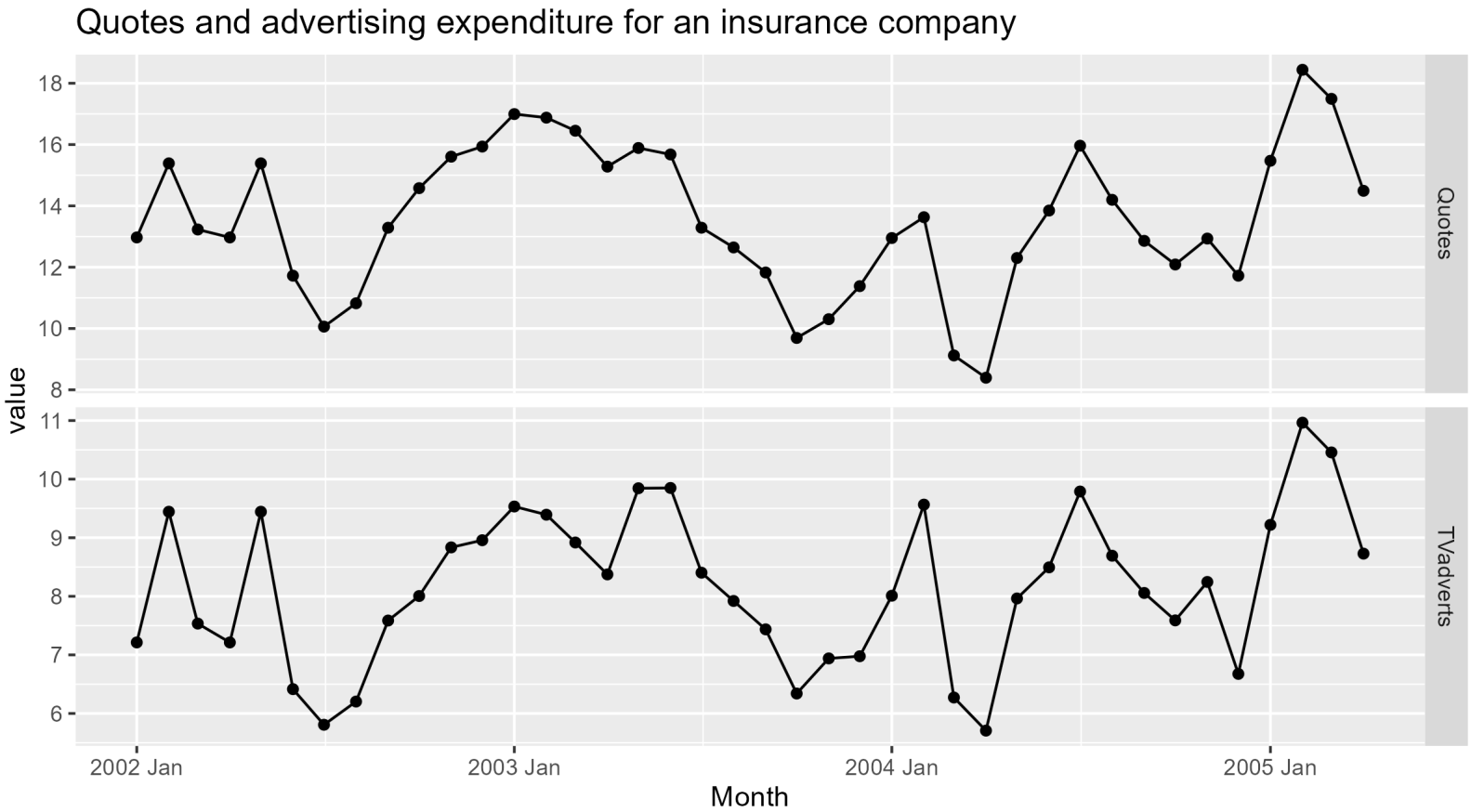
- where η_t is an ARIMA process. How many lagged versions to use, together with the values for p and q for ARIMA can be selected using AICc.

Example I

- We will consider as example a data set, *insurance*, containing information about monthly advertising expenditure and number of quotations for an insurance company between 2002 and 2005.
- Since we use lagged versions of advertisements, we will replace the first 3 Quotes with NA (to make sure that all models use exactly the same training set)

```
insurance |>
  pivot_longer(c(Quotes, TVadverts)) |>
  ggplot(mapping = aes(x = Month, y = value)) +
  geom_line() +
  facet_grid(vars(name), scales = "free_y")
```

Example II


Quotes and advertising expenditure for an insurance company
value
18
16
14
12
10
8
11
10
9
8
7
6
Quotes
TVadverts
2002 Jan
2003 Jan
2004 Jan
2005 Jan
Month

Example III

```
insurance_model <- insurance |>
  mutate(Quotes = c(NA, NA, NA, Quotes[4:40])) |>
  model(
    lag0 = ARIMA(Quotes ~ pdq(d = 0) + TVadverts),
    lag1 = ARIMA(Quotes ~ pdq(d = 0) + TVadverts + lag(TVadverts)),
    lag2 = ARIMA(Quotes ~ pdq(d = 0) + TVadverts + lag(TVadverts) + lag(
     TVadverts, 2)),
    lag3 = ARIMA(Quotes ~ pdq(d = 0) + TVadverts + lag(TVadverts) + lag(
     TVadverts, 2) + lag(TVadverts, 3))
  )

glance(insurance_model) |>
  arrange(AICc)
```

```
  .model sigma2 log_lik   AIC  AICc   BIC ar_roots  ma_roots
   <chr>   <dbl>   <dbl> <dbl> <dbl> <dbl> <list>    <list>
1 lag1    0.209   -24.0  58.1  59.9  66.5 <cpl [1]> <cpl [1]>
2 lag2    0.215   -24.0  60.0  62.6  70.2 <cpl [1]> <cpl [1]>
3 lag3    0.206   -22.2  60.3  65.0  73.8 <cpl [1]> <cpl [1]>
4 lag0    0.265   -28.3  66.6  68.3  75.0 <cpl [2]> <cpl [0]>
```

- The model including a 1 period lag seems to be the best. Let's look at it in more details

Example IV

```
insurance_best <- insurance |>
  model(ARIMA(Quotes ~ pdq(d = 0) + TVadverts + lag(TVadverts, 1)))
insurance_best |> report()
```

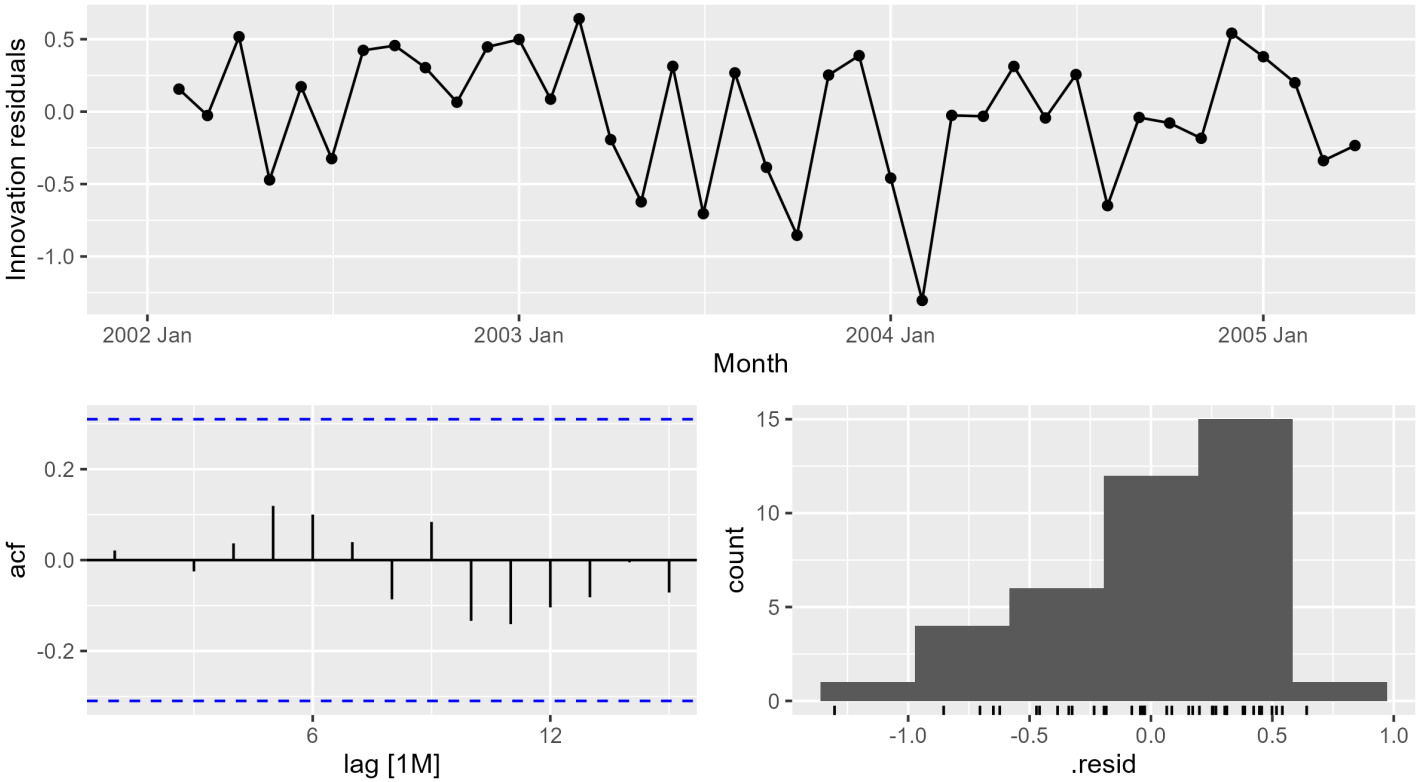
```
Series: Quotes
Model: LM w/ ARIMA(1,0,2) errors

Coefficients:
         ar1     ma1     ma2  TVadverts  lag(TVadverts, 1)  intercept
      0.5123  0.9169  0.4591     1.2527             0.1464     2.1554
s.e.  0.1849  0.2051  0.1895     0.0588             0.0531     0.8595

sigma^2 estimated as 0.2166:  log likelihood=-23.94
AIC=61.88   AICc=65.38   BIC=73.7
```

```
insurance_best |> gg_tsresiduals(lag = 24)
```

Example V


Innovation residuals
0.5
0.0
-0.5
-1.0
2002 Jan
2003 Jan
2004 Jan
2005 Jan
Month
acf
0.2
0.0
-0.2
6
12
lag [1M]
count
15
10
5
0
-1.0
-0.5
0.0
0.5
1.0
.resid

Example VI

```
    best_model |>
  augment() |>
  features(.innov, ljung_box, lag = 24, dof = 3)
```

```
A tibble: 1 × 3
  .model                                      lb_stat lb_pvalue
  <chr>                                         <dbl>     <dbl>
1 ARIMA(Quotes ~ pdq(d = 0) + TVadverts + l...    10.2     0.977
>
```

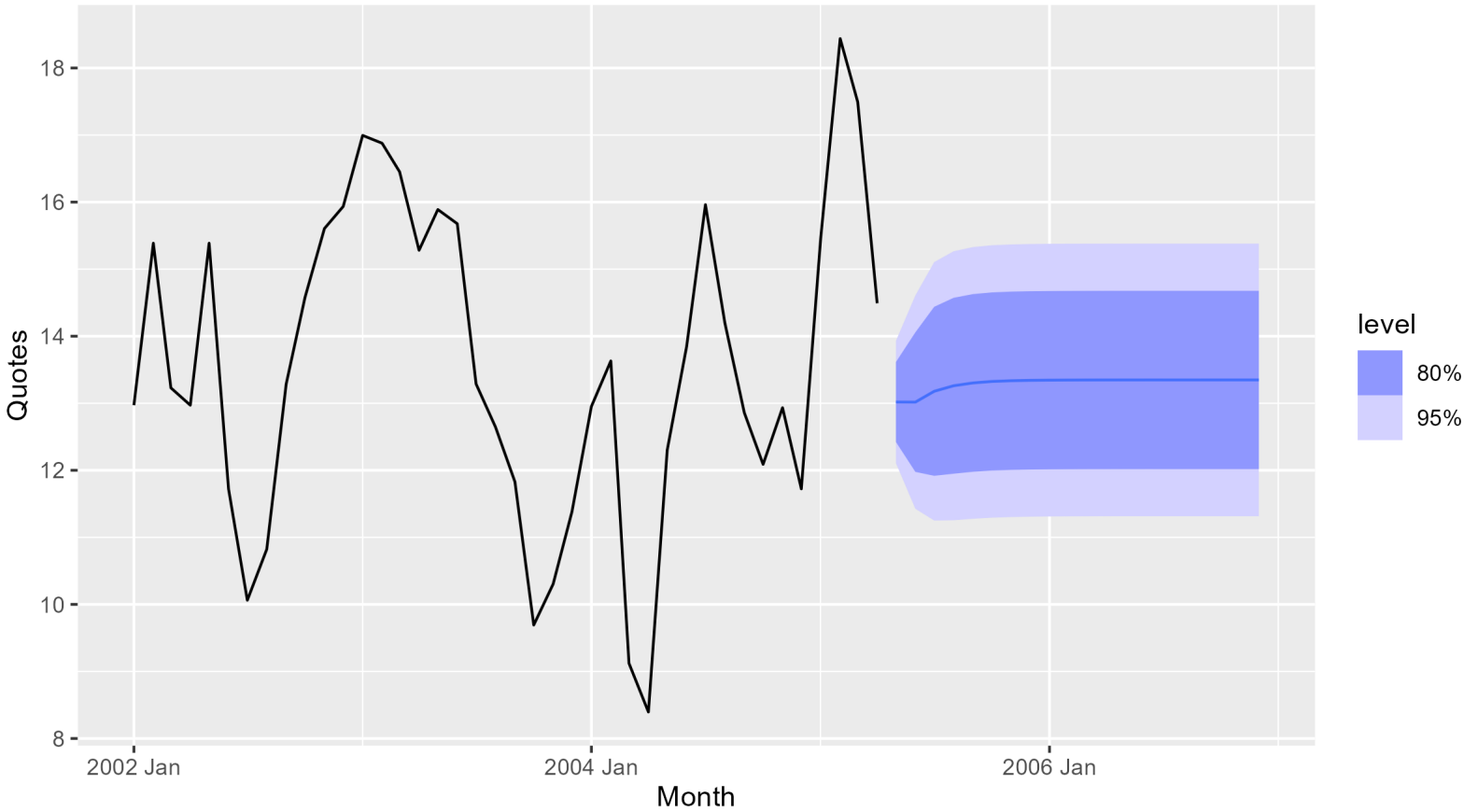
Example VII

- We can use this model for forecasting, if we assign future values for TVadverts. Let's assume it will be 8 for the next 20 months (its mean is 8.17) - this is a scenario-based forecasting, it tell us how many quotes we are likely to get, if we spend a given amount of money on advertisement.

```
insurance_future <- new_data(insurance, 20)
insurance_future <- insurance_future |>
  mutate(TVadverts = 8)

insurance_best |>
  forecast(new_data = insurance_future) |>
  autoplot(insurance) +
```

Example VIII



Dummy variables - recap I

- When we discussed time series regression models we talked about dummy variables: variable we add to represent categorical values or special situations. For example:
  - A variable with values 0 or 1 to denote if a given day is a holiday/weekday
  - A variable with values 0 or 1 to denote if an observation is an outlier or not

Dummy variables - recap II

- For seasonal data we can have dummy variables for the seasons. For example, if we have quarterly data, we need 3 dummy variables. One possible way to define them is the following:
  - Q1 - 0, 0, 0
  - Q2 - 1, 0, 0
  - Q3 - 0, 1, 0
  - Q4 - 0, 0, 1
- But what if, we have long seasonal periods? If you have daily data, and you are having yearly seasonality, that means 364 dummies (and a regression model needs to estimate a coefficient for each).
- Moreover, seasonal ETS and ARIMA were designed for shorter time periods.

Fourier series I

- There is a special case of dummy variables which uses Fourier terms. These variables are particularly useful if you have long periods (think about yearly periodicity in the vic_elec time series) or if you have non-integer periods (weekly data with yearly periods).

- The Fourier terms are pairs of terms, made of a sine term and a cosine term.

$$s_k(t) = sin(\frac{2 * \pi * k * t}{m})$$

$$c_k(t) = cos(\frac{2 * \pi * k * t}{m})$$

- where m is the seasonal period, which can be non-integer (e.g. 52.18 for weekly data).

Fourier series II

- The time series regression expression is then:

$$y_t = a + b * t + \sum_{k=1}^{K} [\alpha_k * s_k(t) + \beta_k * c_k(t)] + \epsilon_t$$

- K is a parameter of the model which should be chosen to minimize AICc.
- For every value of k, a pair of terms will be estimated. The maximum number of such terms is the number of seasons - 1.
- In *R*, you can use the *fourier* special in TSLM to estimate such coefficients.
- Regression models containing Fourier terms are often called **harmonic regression**.

Fourier series III

```
vic_elec |>
    model(TSLM(Demand ~ Temperature + fourier(period = 48, K = 4))) |>
    report()
```

Fourier series IV

```
Series: Demand
Model: TSLM

Residuals:
      Min        1Q    Median        3Q       Max
-2037.826  -380.667    -1.442   371.159  4070.622

Coefficients:
                                Estimate Std. Error t value Pr(>|t|)
(Intercept)                     4543.775      9.267 490.301  < 2e-16 ***
Temperature                        7.480      0.543  13.774  < 2e-16 ***
fourier(period = 48, K = 4)C1_48  231.495      3.985  58.091  < 2e-16 ***
fourier(period = 48, K = 4)S1_48  683.120      4.313 158.378  < 2e-16 ***
fourier(period = 48, K = 4)C2_48  218.337      3.968  55.027  < 2e-16 ***
fourier(period = 48, K = 4)S2_48 -249.037      3.985 -62.495  < 2e-16 ***
fourier(period = 48, K = 4)C3_48   55.890      3.968  14.086  < 2e-16 ***
fourier(period = 48, K = 4)S3_48 -142.430      3.968 -35.897  < 2e-16 ***
fourier(period = 48, K = 4)C4_48 -108.622      3.968 -27.375  < 2e-16 ***
fourier(period = 48, K = 4)S4_48   26.441      3.968   6.664 2.69e-11 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 643.5 on 52598 degrees of freedom
Multiple R-squared: 0.4583, Adjusted R-squared: 0.4583
F-statistic:  4945 on 9 and 52598 DF, p-value: < 2.22e-16
```

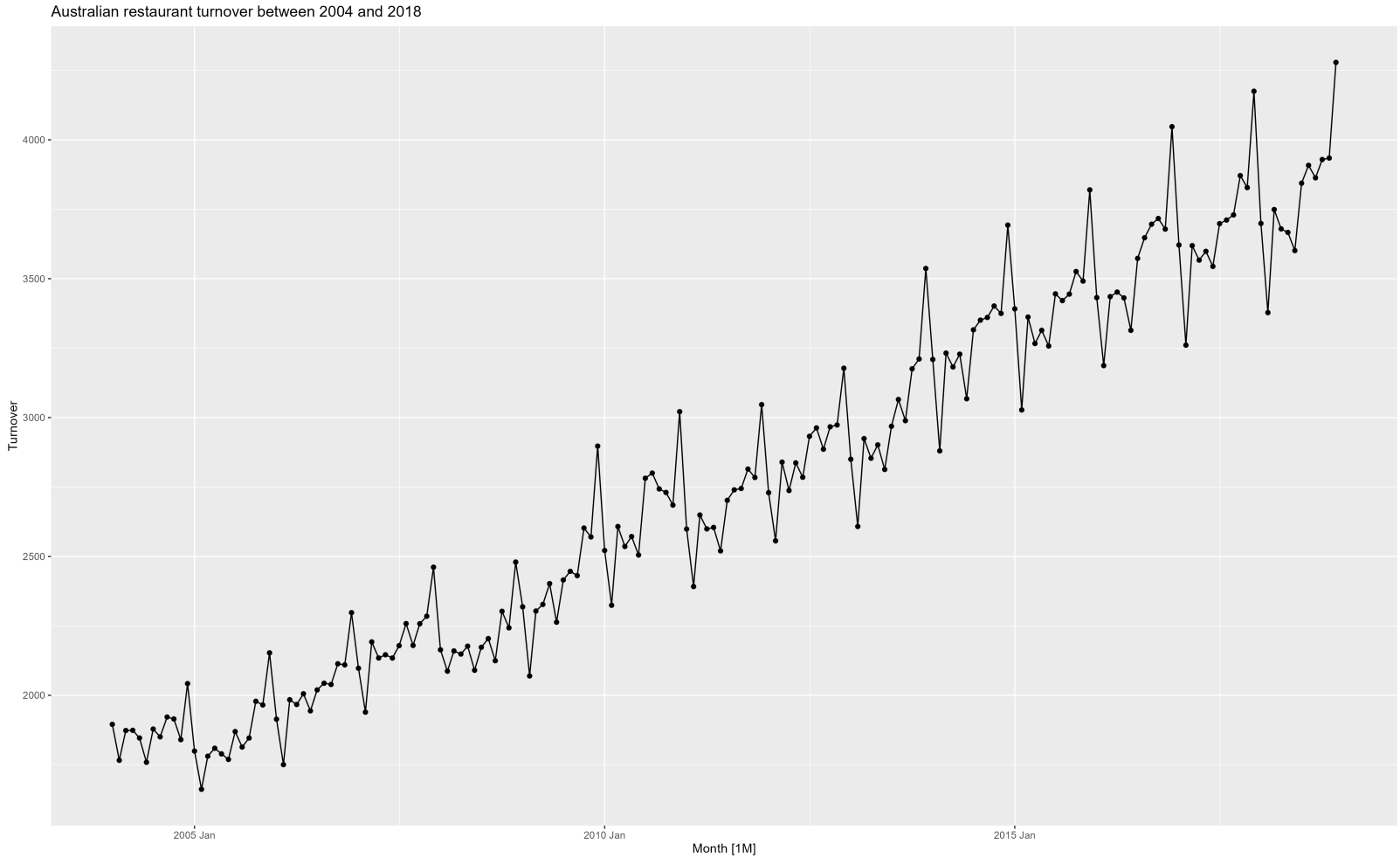
Another example I

- Let's consider another example, the eating out expenditure from the Australian retail (*aus_retail*) time series.
- It is monthly data, so we can have at most 11 seasonal dummies. We will build 6 models with different values of K for the fourier terms (1 to 6) and plot the data, the fitted values and forecasts for the next 2 years.

Another example II

```
aus_cafe <- aus_retail |>
  filter(Industry == "Cafes, restaurants and takeaway food services",
         year(Month) >= 2004, year(Month) <= 2018) |>
  summarise(Turnover = sum(Turnover))
aus_cafe
aus_cafe |> autoplot() +
  geom_point() +
   labs(title = "Australian restaurant turnover between 2004 and 2018")
```

Another example III



Another example IV

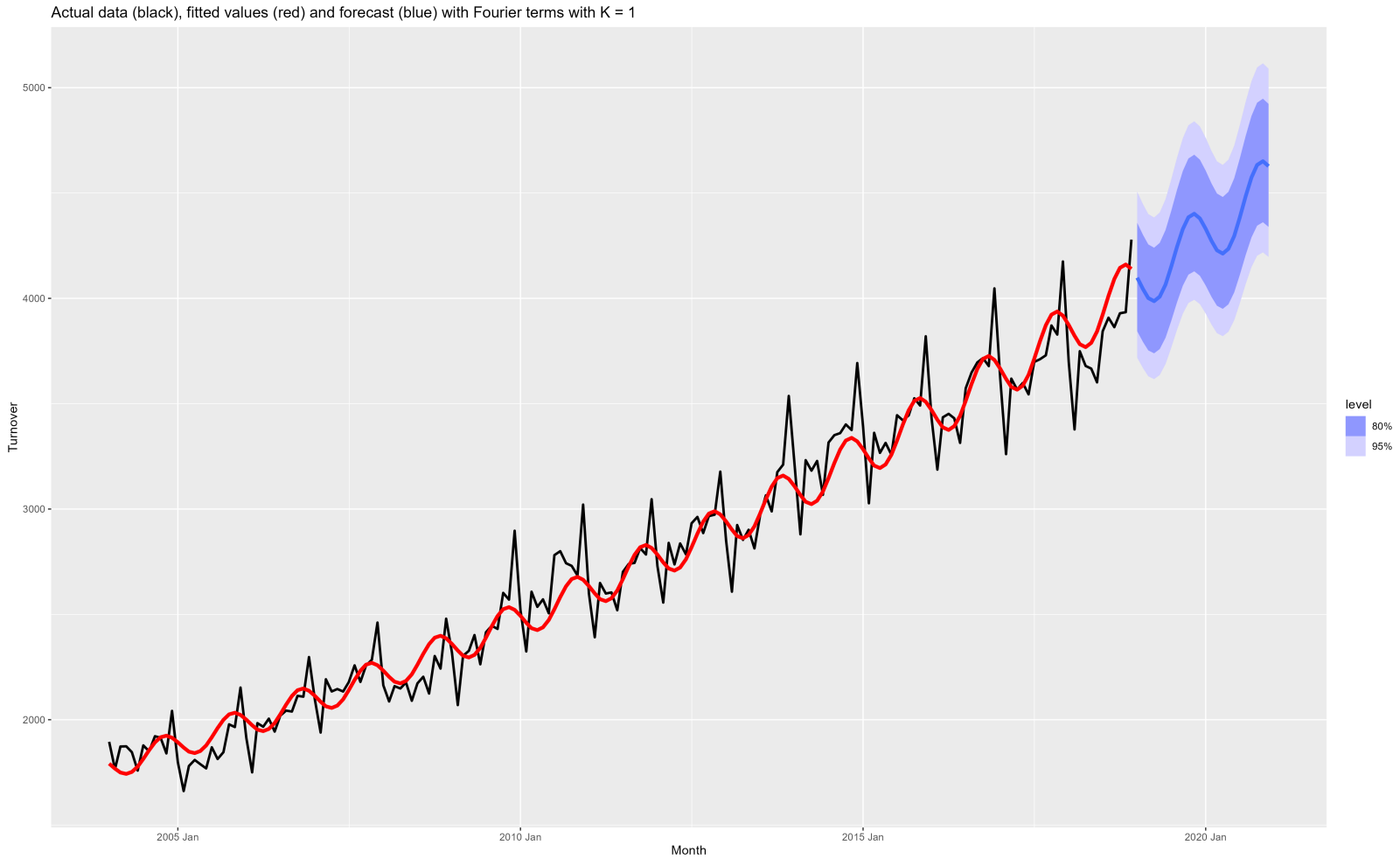
```
cafe_models <- aus_cafe |>
  model(
    K1 = TSLM(log(Turnover) ~ trend() + fourier(K = 1)),
    K2 = TSLM(log(Turnover) ~ trend() + fourier(K = 2)),
    K3 = TSLM(log(Turnover) ~ trend() + fourier(K = 3)),
    K4 = TSLM(log(Turnover) ~ trend() + fourier(K = 4)),
    K5 = TSLM(log(Turnover) ~ trend() + fourier(K = 5)),
    K6 = TSLM(log(Turnover) ~ trend() + fourier(K = 6)),
  )

cafe_models |>
  forecast(h = 24) -> cafe_forecast

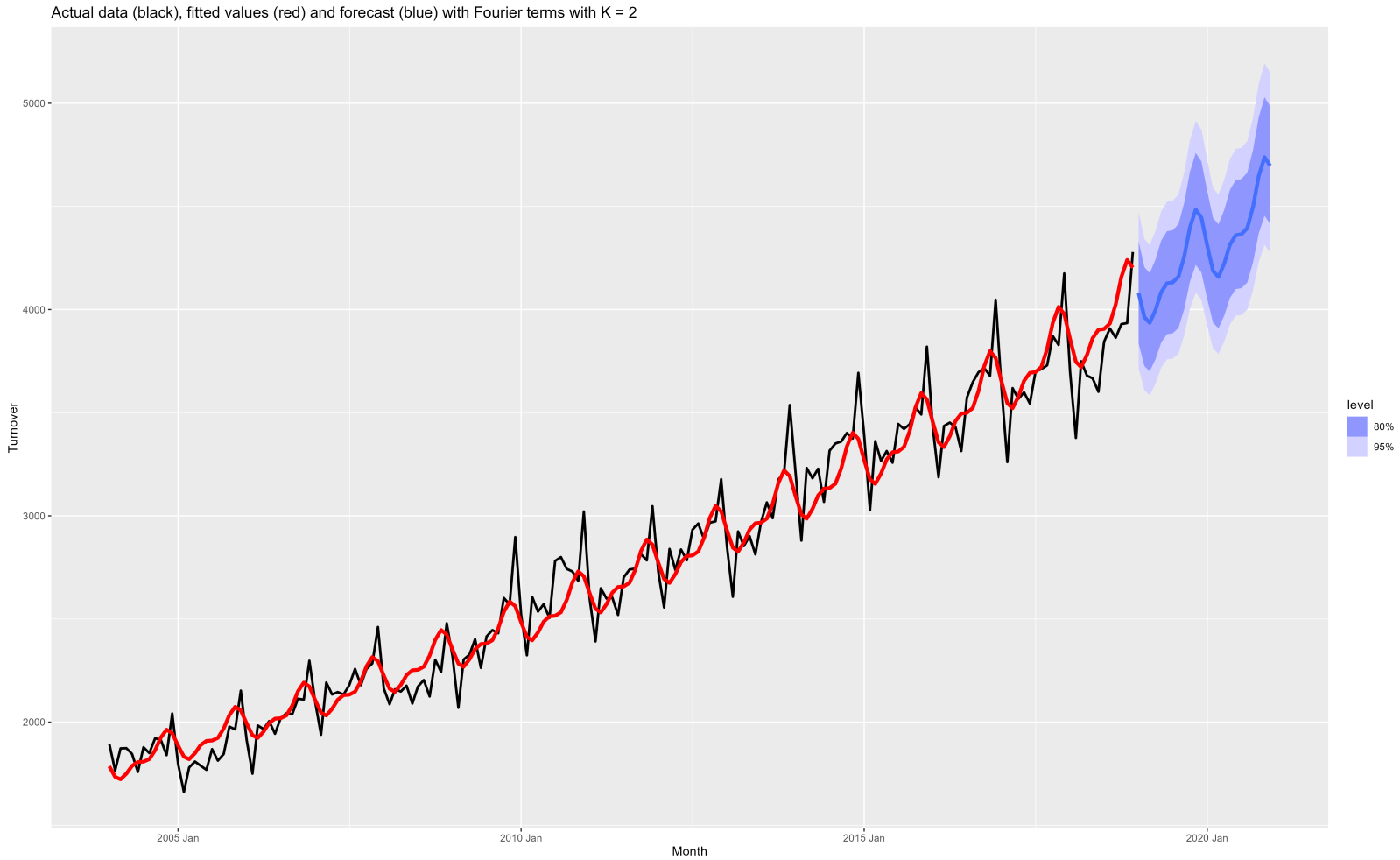
cafe_models |>
  augment() -> cafe_augmented

cafe_forecast |>
  filter(.model == "K1") |> #experiment with other models
  autoplot(linewidth = 1.5) +
  autolayer(aus_cafe, Turnover, linewidth = 1) +
  autolayer(cafe_augmented |> filter(.model == "K1"), .fitted, color= "red",
     linewidth = 1.5) +
  labs(title = "Actual data (black), fitted values (red) and forecast (blue)
     with Fourier terms with K = 1")
```

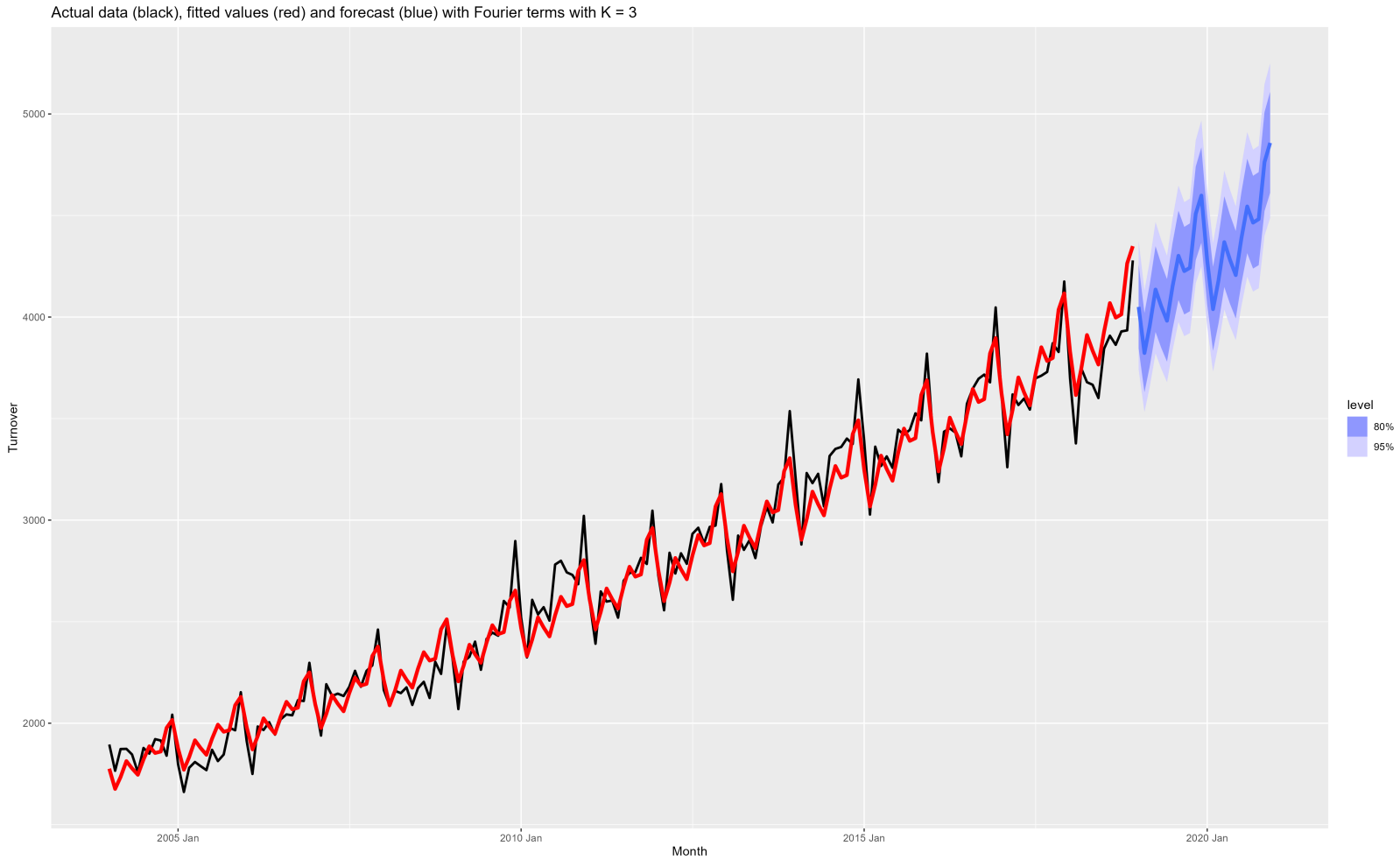
Another example V



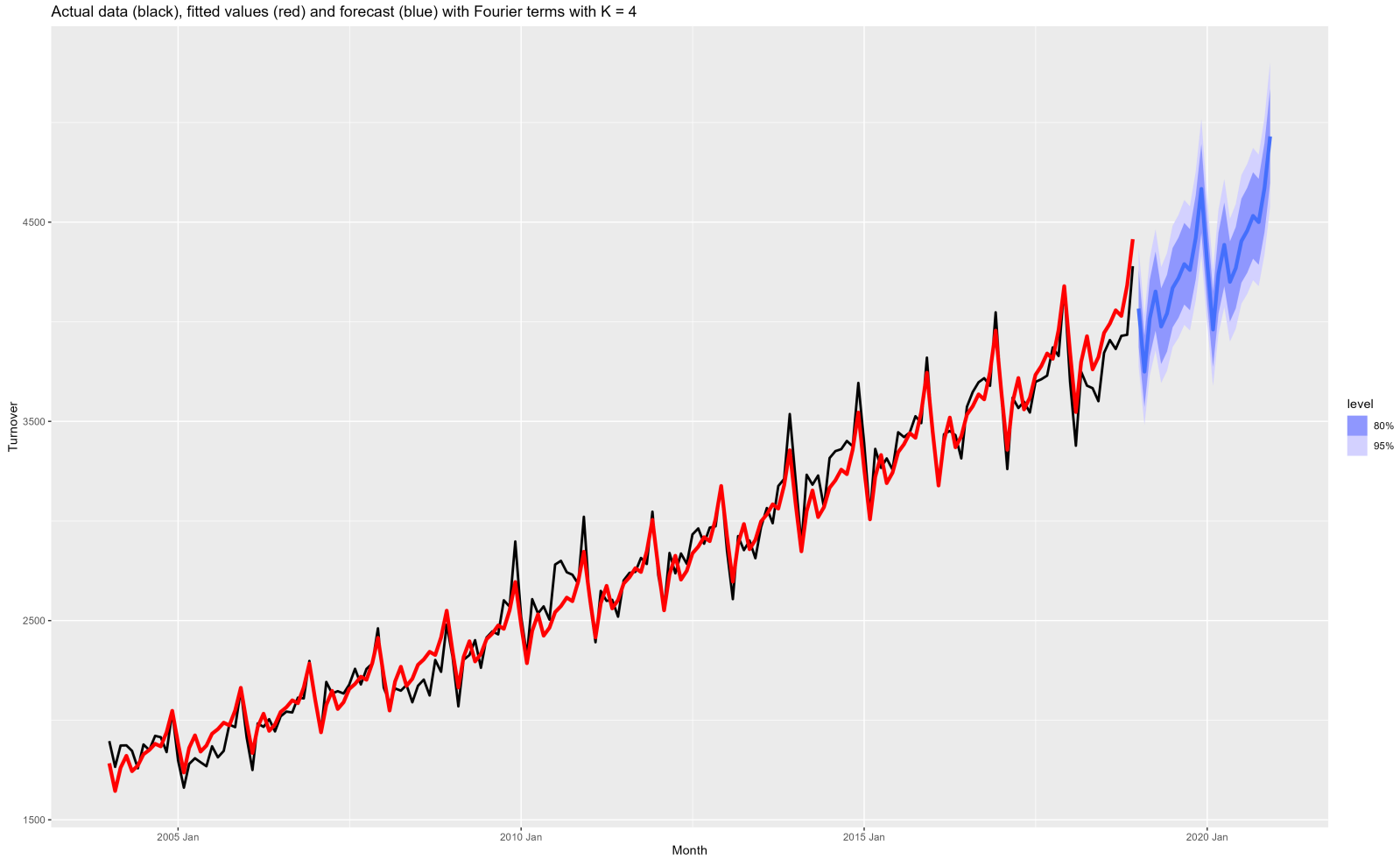
Another example VI



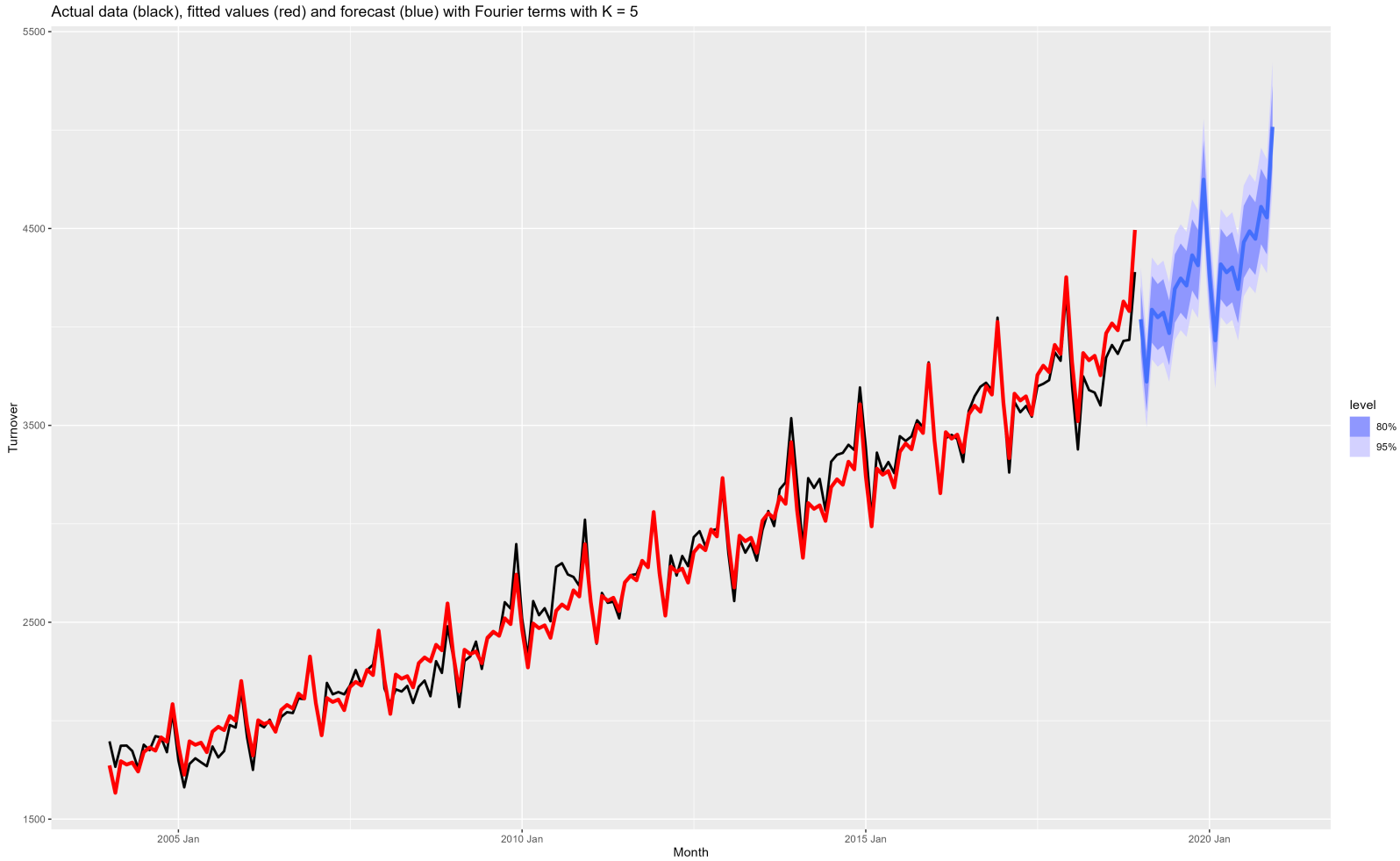
Another example VII



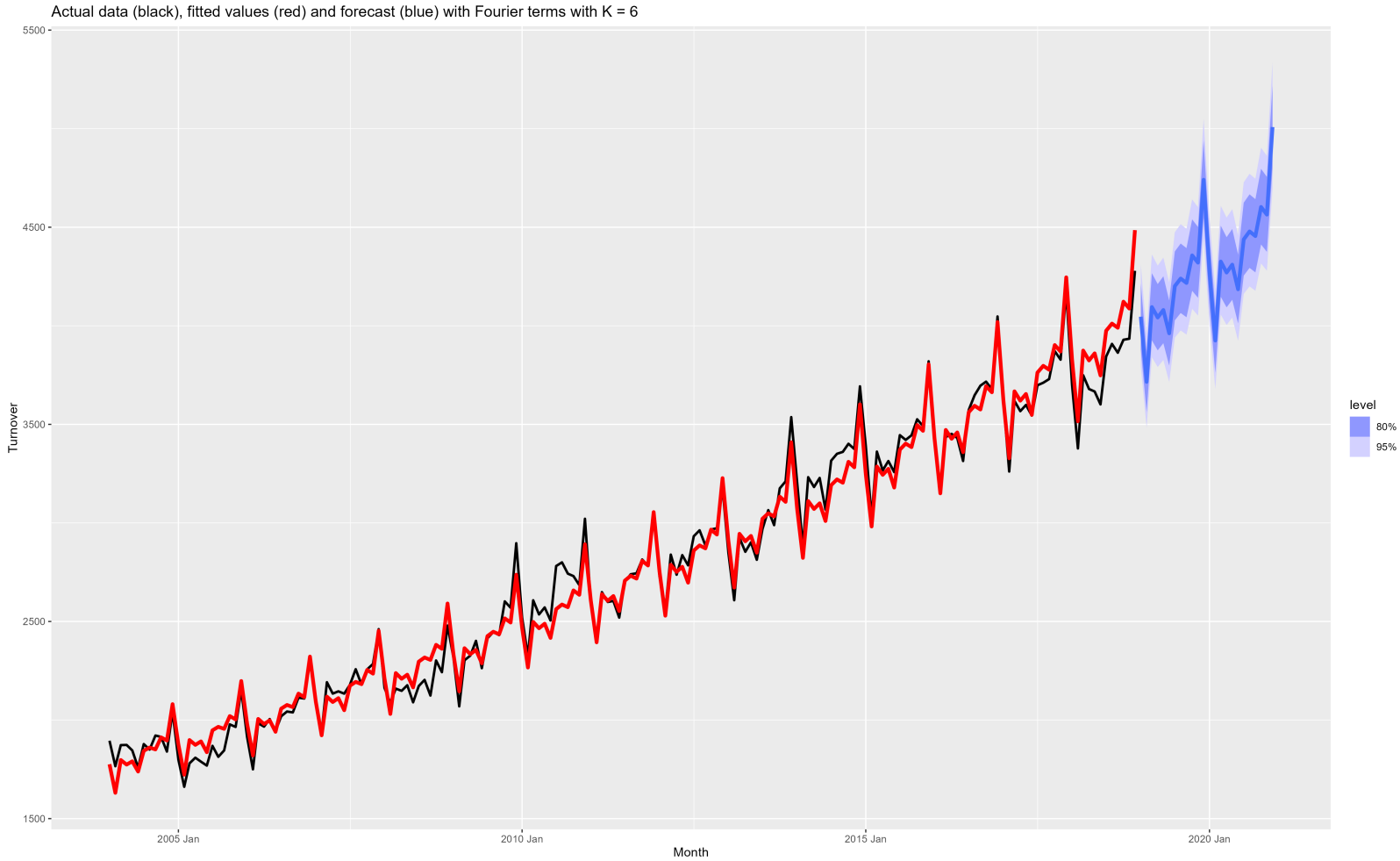
Another example VIII



Another example IX



Another example X



Another example XI

- Which model is the best?

```
  cafe_models |>
glance() |>
select(.model, AIC, AICc) |>
arrange(AICc)
```

```
  .model    AIC   AICc
  <chr>   <dbl>  <dbl>
1 K5     -1236. -1234.
2 K6     -1235. -1232.
3 K4     -1185. -1183.
4 K3     -1161. -1160.
5 K2     -1100. -1099.
6 K1     -1086. -1085.
```

Dynamic harmonic regression I

- Harmonic regression can be a part of dynamic regression models as well. Let's see the previous example again, but this time fit a regression model with ARIMA errors.

Dynamic harmonic regression II

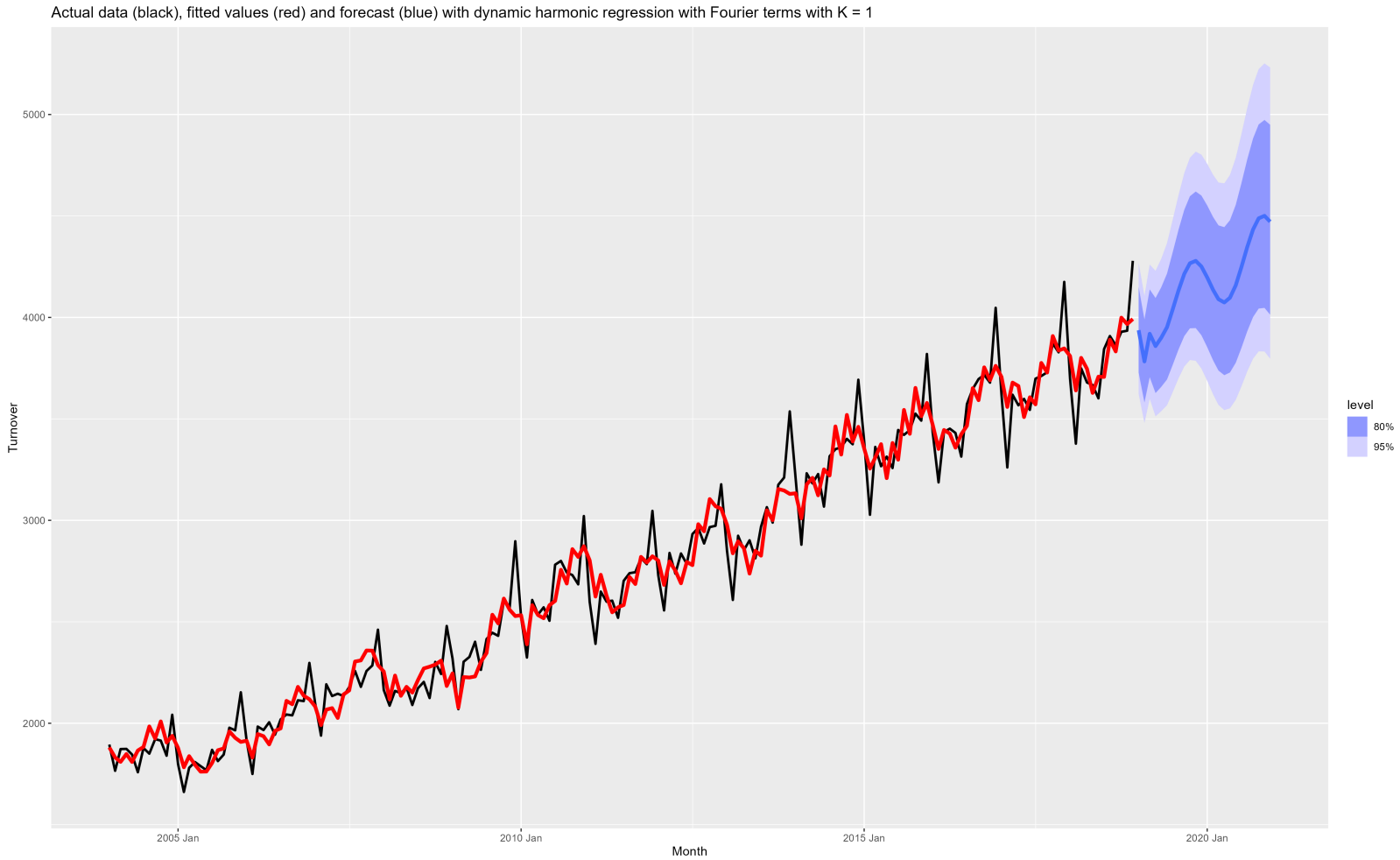
```
cafe_models <- aus_cafe |>
  model(
    K1 = ARIMA(log(Turnover) ~ fourier(K = 1) + PDQ(0, 0, 0)),
    K2 = ARIMA(log(Turnover) ~ fourier(K = 2) + PDQ(0, 0, 0)),
    K3 = ARIMA(log(Turnover) ~ fourier(K = 3) + PDQ(0, 0, 0)),
    K4 = ARIMA(log(Turnover) ~ fourier(K = 4) + PDQ(0, 0, 0)),
    K5 = ARIMA(log(Turnover) ~ fourier(K = 5) + PDQ(0, 0, 0)),
    K6 = ARIMA(log(Turnover) ~ fourier(K = 6) + PDQ(0, 0, 0)),
  )

cafe_models |>
  forecast(h = 24) -> cafe_forecast

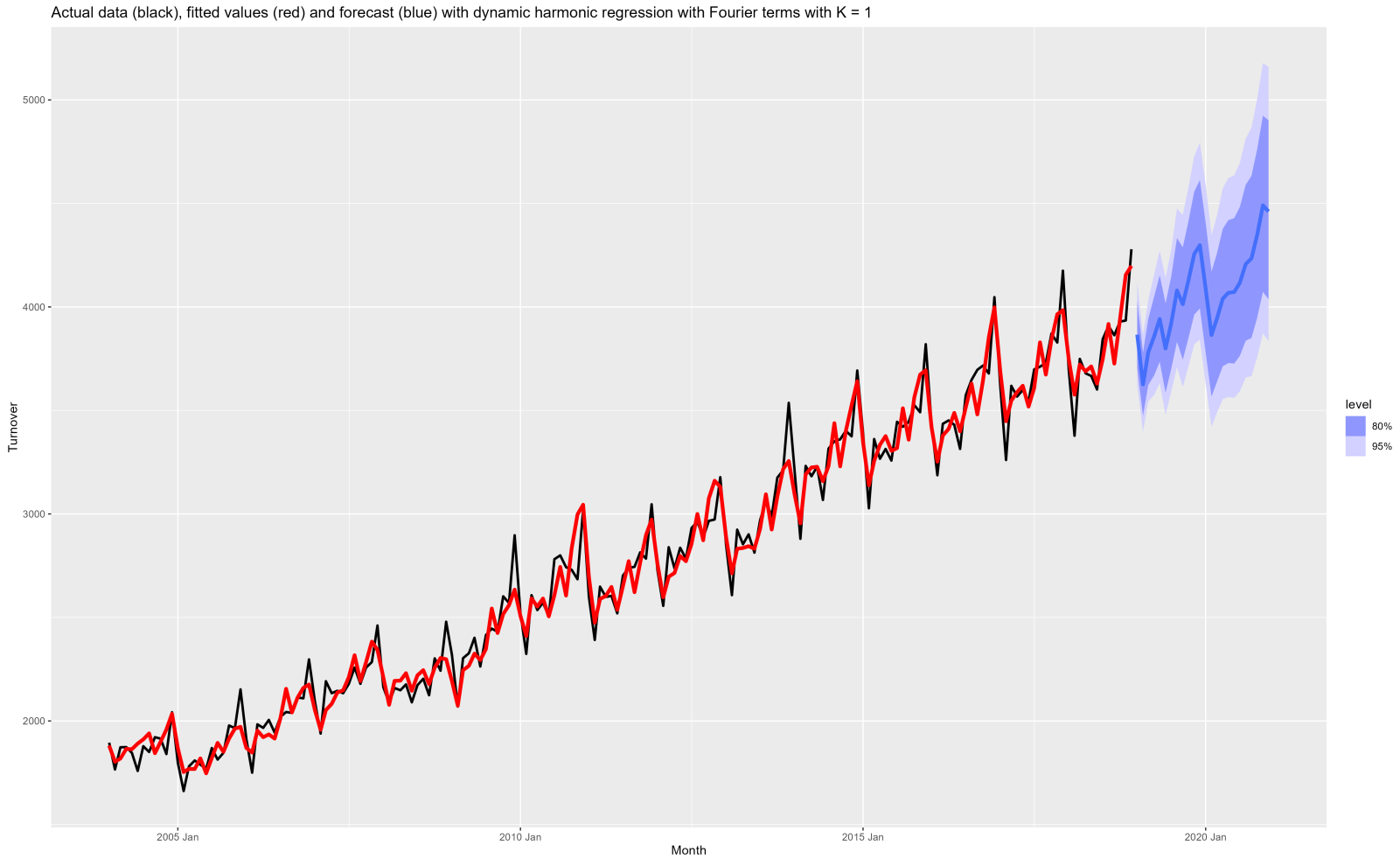
cafe_models |>
  augment() -> cafe_augmented

cafe_forecast |>
  filter(.model == "K1") |> #experiment with other models
  autoplot(linewidth = 1.5) +
  autolayer(aus_cafe, Turnover, linewidth = 1) +
  autolayer(cafe_augmented |> filter(.model == "K1"), .fitted, color= "red",
     linewidth = 1.5) +
  labs(title = "Actual data (black), fitted values (red) and forecast (blue)
     with dynamic harmonic regression with Fourier terms with K = 1")
```

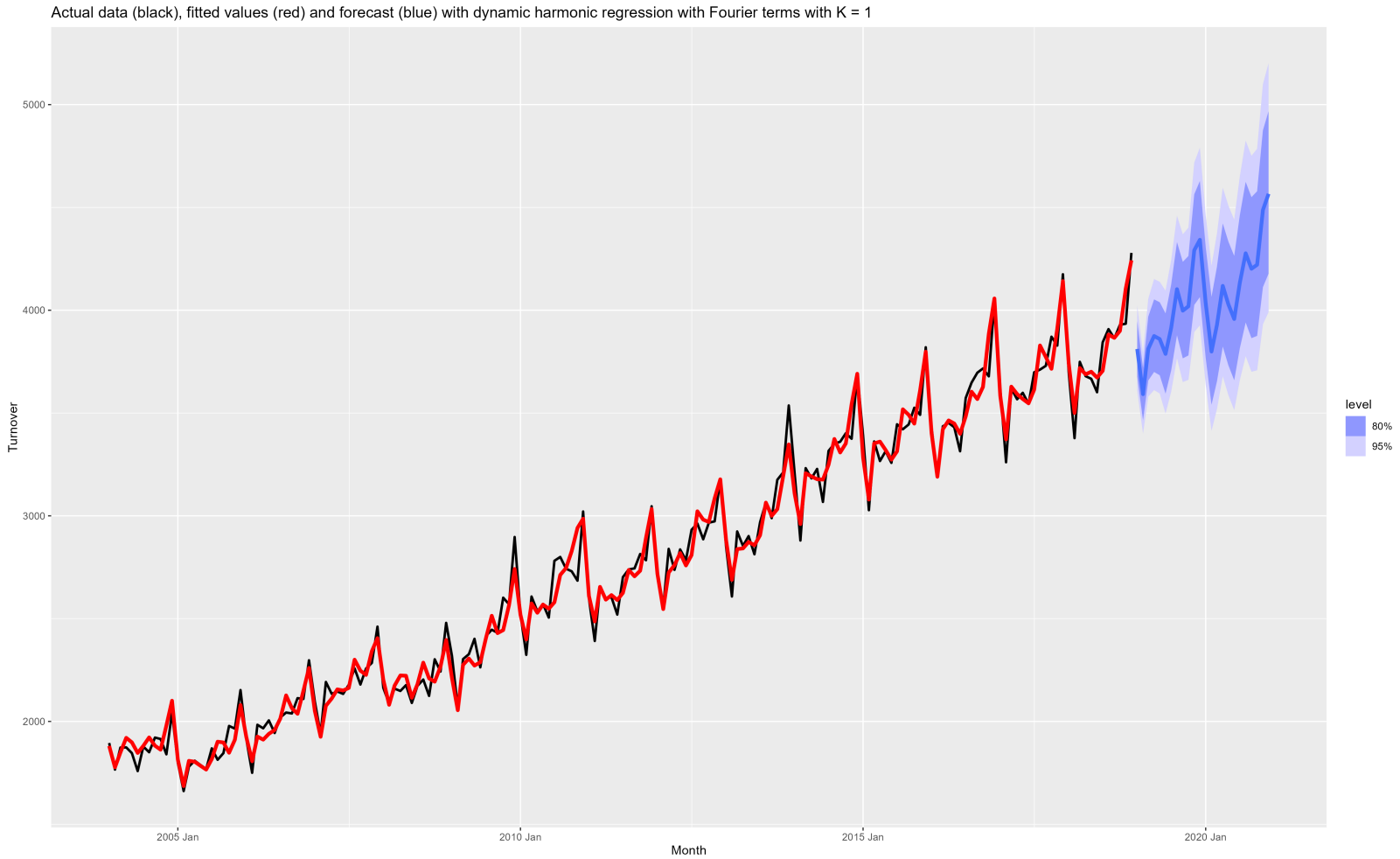
Dynamic harmonic regression III



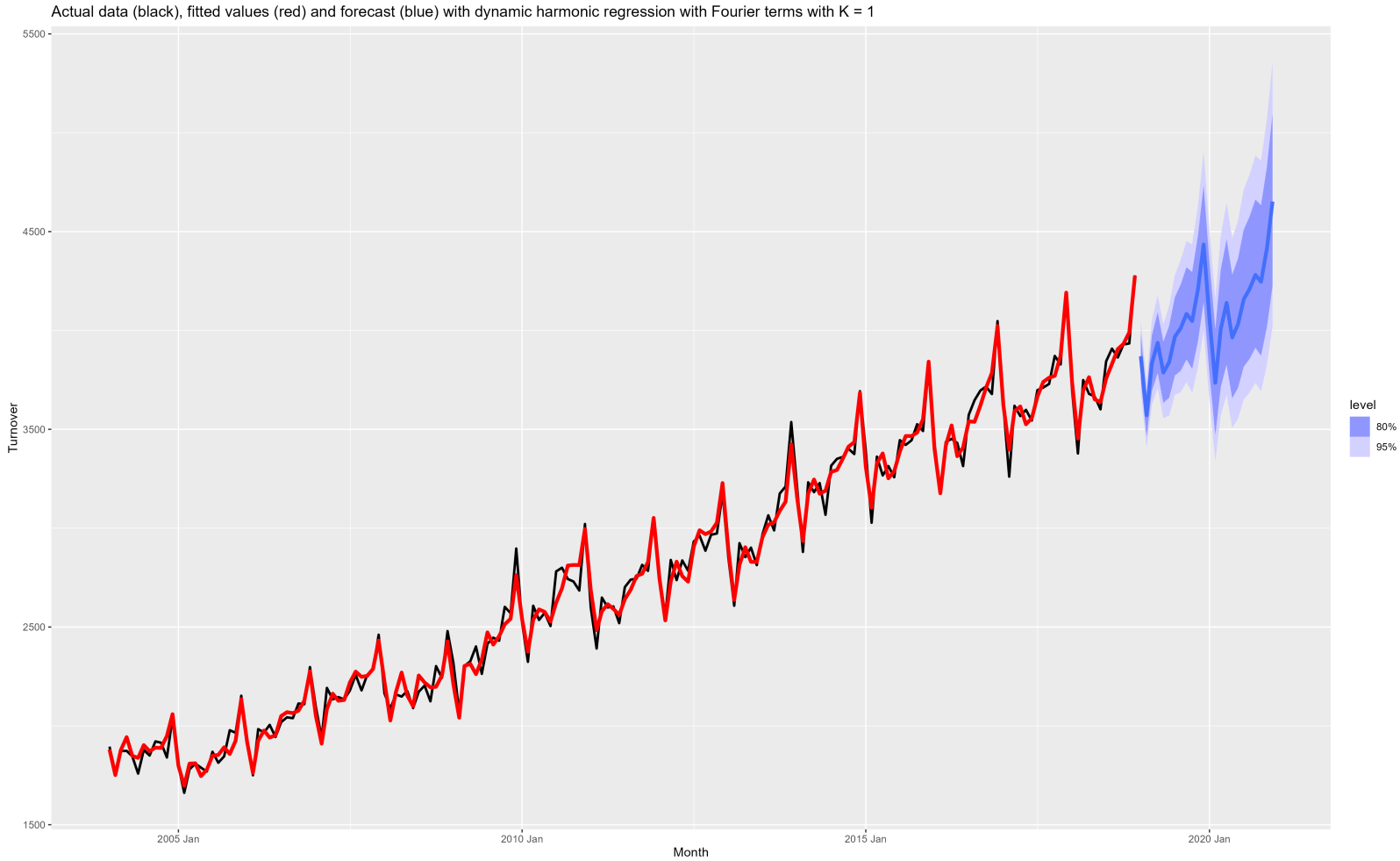
Dynamic harmonic regression IV



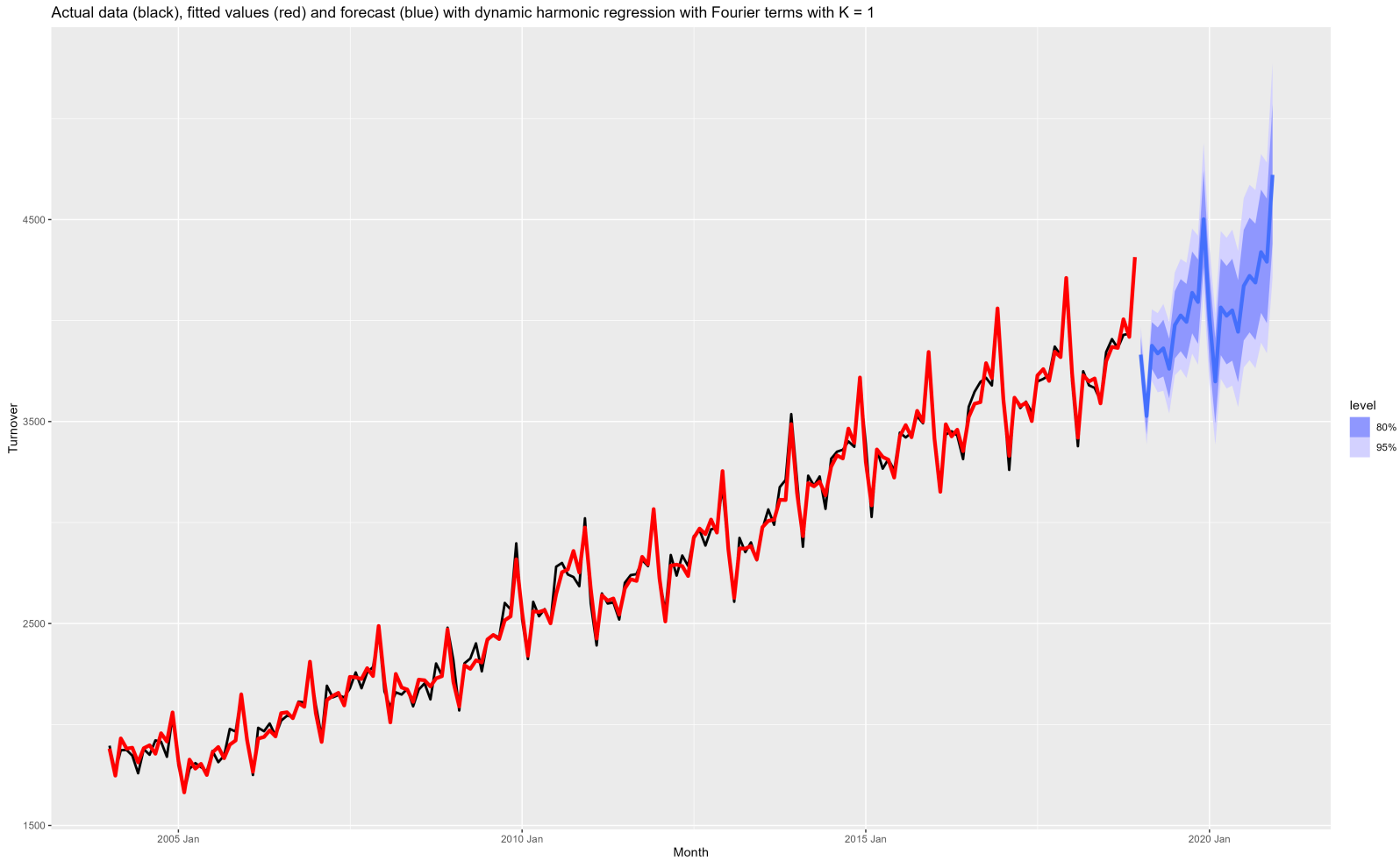
Dynamic harmonic regression V



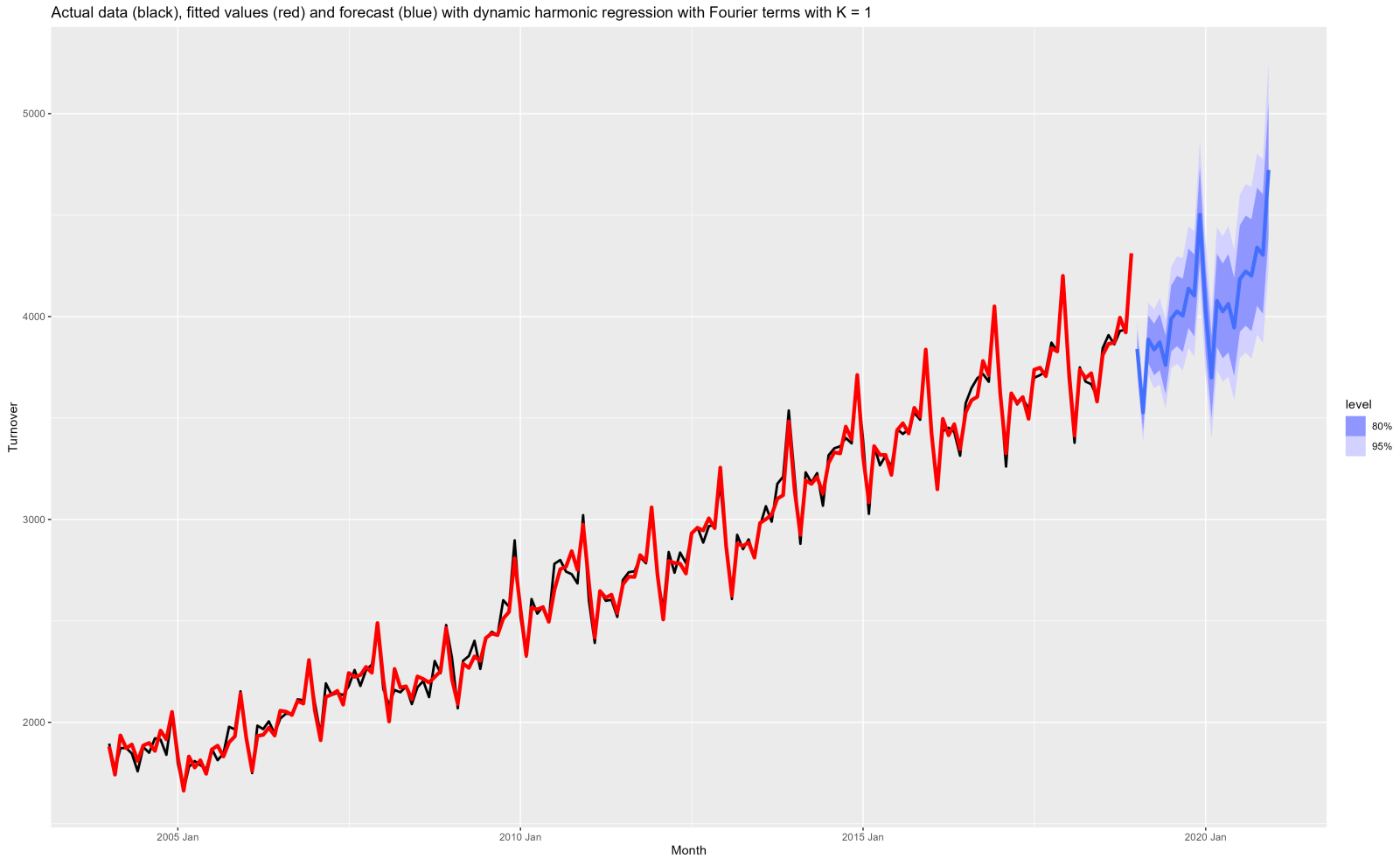
Dynamic harmonic regression VI



Dynamic harmonic regression VII



Dynamic harmonic regression VIII



Dynamic harmonic regression IX

- Which model is the best?

```
  cafe_models |>
glance() |>
select(.model, AIC, AICc) |>
arrange(AICc)
```

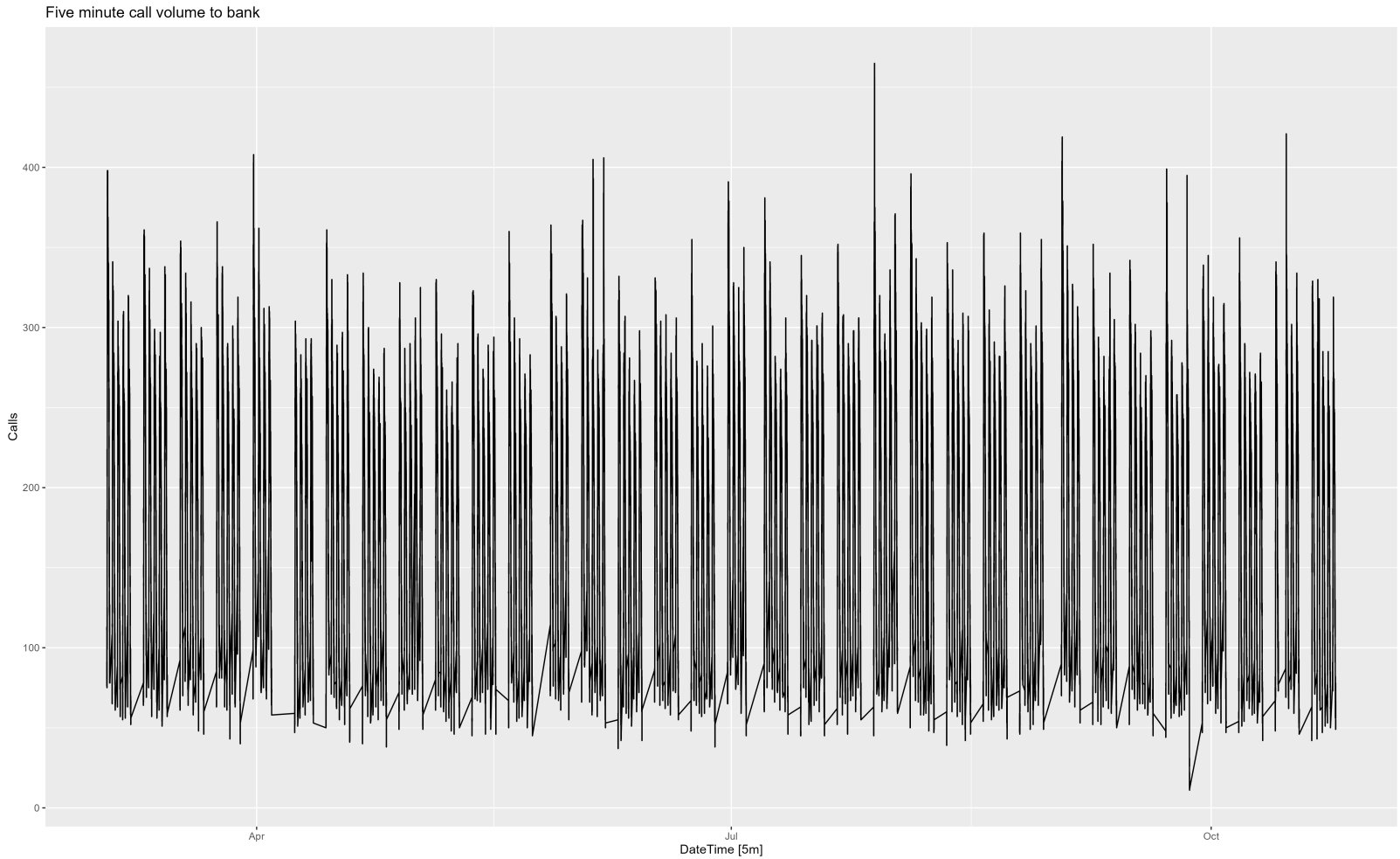
```
  .model   AIC  AICc
  <chr>  <dbl> <dbl>
1 K6     -920. -918.
2 K5     -919. -917.
3 K4     -822. -818.
4 K3     -763. -761.
5 K2     -700. -698.
6 K1     -616. -615.
```

Time series with complex seasonality I

- Most of the time series that we have worked with were either monthly or quarterly and they had a yearly seasonality.
- However, data with higher frequency (daily, hourly, etc.) might present more complex seasonality. For example:
  - Daily data can have a weekly and a yearly seasonality
  - Hourly data can have a daily, a weekly and a yearly seasonality.
- Let's look at a time series which shows the number of calls to a North American bank between 7:00 am and 9:05 pm for every 5 minutes. We only have data for weekdays, for a 33 week period.

```
  bank_calls |>
autoplot() +
labs(title = "Five minute call volume to bank")
```

Time series with complex seasonality II



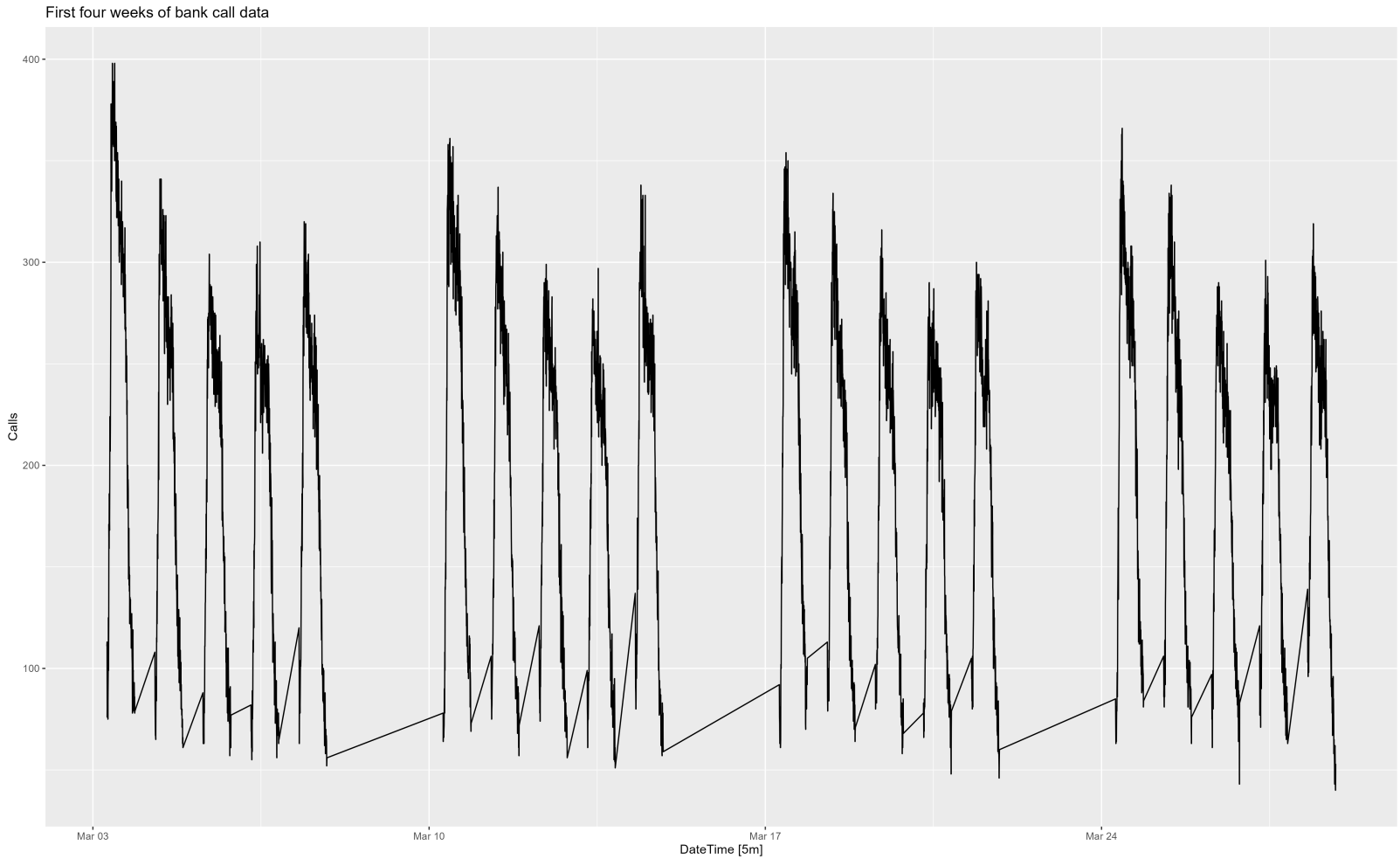
Time series with complex seasonality III

- Since frequency is 5 minutes, we have 12 observations per hour and 14 hours a day + the last observation (9:05) $\rightarrow$ 169 observations per day.

- Since the data is quite dense, let us look at the first 4 weeks only

```
bank_calls |>
  head(n = 169 * 20) |>
  autoplot() +
  labs(title = "First four weeks of bank call data")
```

Time series with complex seasonality IV



Time series with complex seasonality V

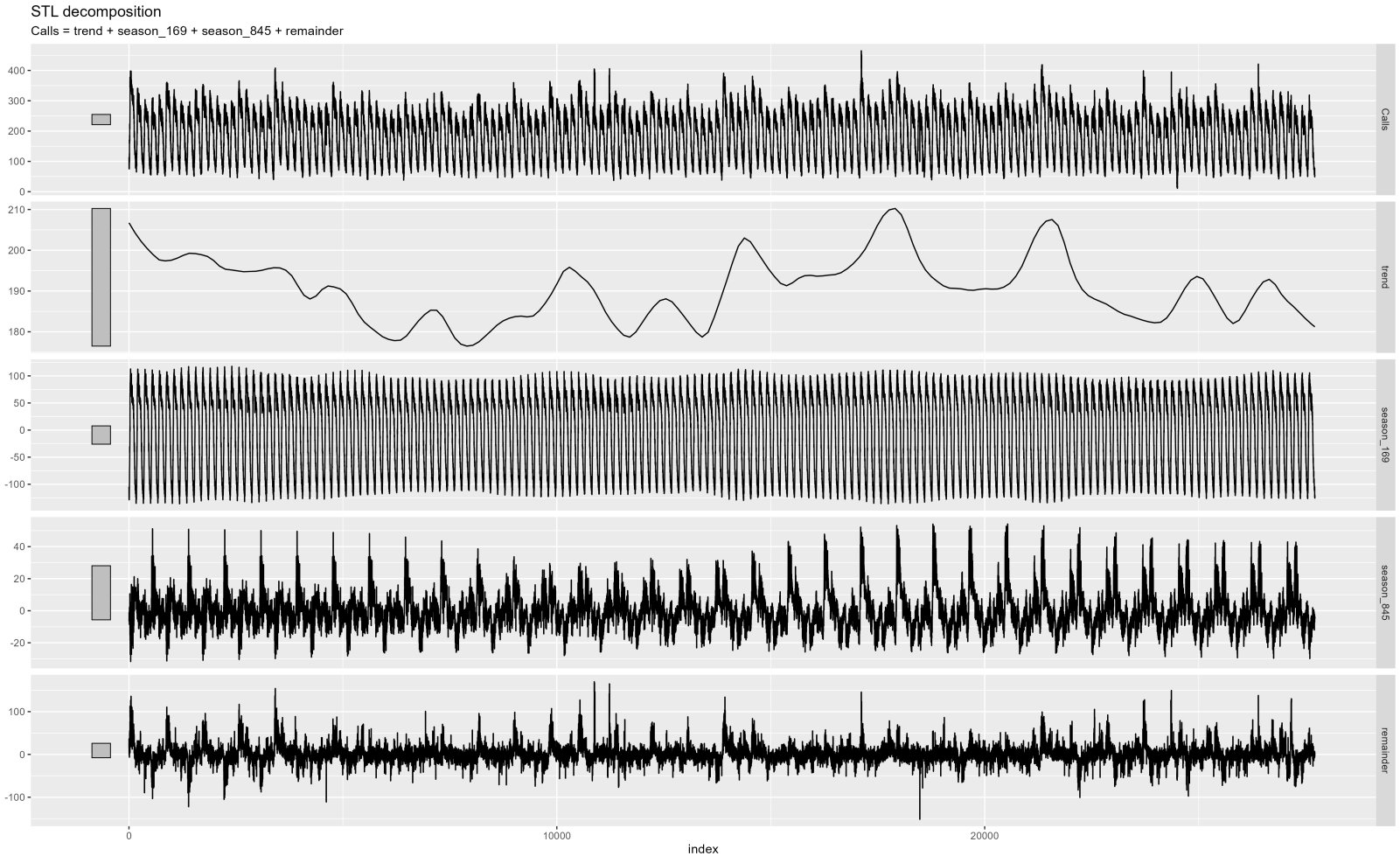
- For such a time series, we might have daily and weekly seasonality (we could have yearly seasonality as well, but since we only have 33 weeks of data, we cannot test that).
- STL decomposition can deal with multiple seasonal components.
- However, we cannot directly apply STL to the data, since we have missing timestamps (we only have workdays). So we will reindex the data (with simple numerical indexes) and then pass the seasonal frequency *manually* to STL.

Time series with complex seasonality VI

```
calls <- bank_calls |>
  mutate(index = row_number()) |>
  update_tsibble(index = index, regular = TRUE)

calls |>
  model(stl = STL(Calls ~ season(period = 169) + season(period = 169 * 5),
      robust = TRUE)) |>
  components() |>
  autoplot()
```

Time series with complex seasonality VII



Time series with complex seasonality VIII

- We can see the two seasonal patterns, however, the weekly seasonality is not that strong and trend is not significant either.
- We can use the STL decomposition in a decomposition model. We can try to forecast the seasonal components using a seasonal naive method and we can use ETS for the trend.

Time series with complex seasonality IX

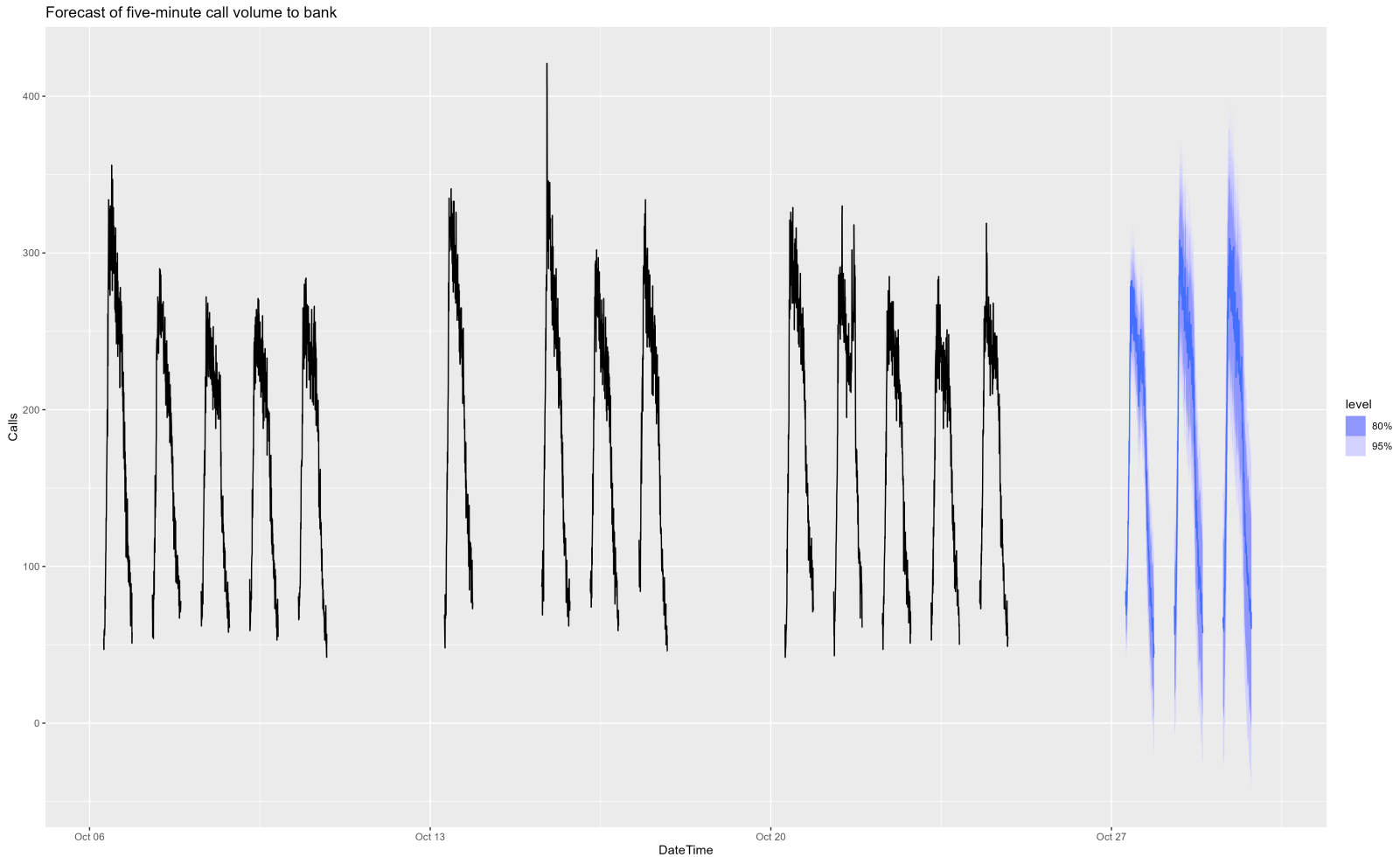
```
    decomp_model <- decomposition_model(
  STL(Calls ~ season(period = 169) +
        season(period = 5*169),
      robust = TRUE),
  ETS(season_adjust ~ season("N"))
)

calls_forecast <- calls |>
  model(decomp_model) |>
  forecast(h = 5 * 169)

#to plot prperly, let s put back the timestamps
forecast_with_time_index <- bank_calls |>
  new_data(n = 7* 24 * 12) |>
  mutate(time = format(DateTime, format = "%H:%M:%S")) |>
  filter(
    time %in% format(bank_calls$DateTime, format = "%H:%M:%S"),
    wday(DateTime, week_start = 1) <= 5) |>
  mutate(index = row_number() + max(calls$index)) |>
  left_join(calls_forecast, by = "index") |>
  as_fable(response = "Calls", distribution = Calls)

forecast_with_time_index |>
  fill_gaps() |>
  autoplot(bank_calls |>tail(n = 14*169) |> fill_gaps()) +
  labs(title = "Forecast of five-minute call volume to bank")
```

Time series with complex seasonality X



Time series with complex seasonality XI

- We can notice an interesting thing: the lower values of the prediction intervals are below zero. Mathematically this is OK, but for the problem, it makes no sense, you cannot have a negative amount of phone calls.

- In such situations, it actually makes more sense to do a square root transformation on the data, to make sure that the back transformed data is always positive (the square of a value is always positive).

Time series with complex seasonality XII

- Let us build another model, this time using Fourier terms. We need Fourier terms for every seasonal period. We will use 10 terms for daily seasonality and 5 for the weekly one.
- We will fit a dynamic regression model with ARIMA errors. This time we will use the square root transformation.

Time series with complex seasonality XIII

```
calls_model2 <- calls |>
  model(
    dhr = ARIMA(sqrt(Calls) ~ PDQ(0, 0, 0) + pdq(d = 0) +
                  fourier(period = 169, K = 10) +
                  fourier(period = 5*169, K = 5)))

calls_forecast2 <- calls_model2 |> forecast(h = 5 * 169)

# Add correct time stamps to fable
forecast_with_time_index2 <- bank_calls |>
  new_data(n = 7 * 24 * 12) |>
  mutate(time = format(DateTime, format = "%H:%M:%S")) |>
  filter(
    time %in% format(bank_calls$DateTime, format = "%H:%M:%S"),
    wday(DateTime, week_start = 1) <= 5
  ) |>
  mutate(index = row_number() + max(calls$index)) |>
  left_join(calls_forecast2, by = "index") |>
  as_fable(response = "Calls", distribution = Calls)

# Plot results with last 3 weeks of data
forecast_with_time_index2 |>
  fill_gaps() |>
  autoplot(bank_calls |> tail(14 * 169) |> fill_gaps()) +
  labs(title = "Five-minute call volume to bank")
```

Time series with complex seasonality XIV

